Graph leetcode problems
23 October 2026

GRAPH LEETCODE PROBLEMS

Problem statement, approach, C++ code, dry run and analysis for every graph problem.

Contents

1. **Number of Provinces** (LeetCode 547)
2. **Number of Islands** (LeetCode 200)
3. **Flood Fill** (LeetCode 733)
4. **Rotten Oranges** (LeetCode 994)
5. **01 Matrix** (LeetCode 542)
6. **Bipartite Graph** (LeetCode 785)
7. **Eventual Safe Nodes** (LeetCode 802)
8. **Course Schedule** (LeetCode 207)
9. **Network Delay Time** (LeetCode 743)
10. **Cheapest Flights Within K Stops** (LeetCode 787) - Constrained Dijkstra
11. **Word Ladder** (LeetCode 127) - BFS Shortest Path
12. **Number of Ways to Arrive at Destination** (LeetCode 1976) - Dijkstra + Dynamic Programming
13. **Shortest Path in Binary Matrix** (LeetCode 1091) - BFS on 2D Grid
14. **Find the City With the Smallest Number of Neighbors** (LeetCode 1334) - Floyd-Warshall vs. Multi-source Dijkstra
15. **Min Cost to Connect Points** (LeetCode 1584) - Prim's Algorithm
16. **Min Cost to Connect Points** (LeetCode 1584) - Kruskal's Algorithm
17. **Number of Operations to Make Network Connected** (LeetCode 1319) - Disjoint Set Union (DSU)
18. **Strongly Connected Components (Kosaraju's Algorithm)** (GeeksforGeeks) - Topological Sort

NUMBER OF PROVINCES

Problem Statement:

547. Number of Provinces

Medium | LeetCode | Open the full problem

There are `n` cities. Some are directly connected to each other and some are not. Connectivity is transitive: if city `a` is directly connected to city `b`, and `b` is directly connected to city `c`, then `a` is indirectly connected to `c`.

A **province** is a group of cities that are directly or indirectly connected to one another, with no other city outside the group connected to any of them.

You are given an `n x n` matrix `isConnected`, where `isConnected[i][j] = 1` if the `i`-th and `j`-th cities are directly connected, and `isConnected[i][j] = 0` otherwise.

Return the total number of provinces.

Example 1:

```
Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
Output: 2
```

Example 2:

```
Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]
Output: 3
```

Constraints:

- `1 <= n <= 200`
- `n == isConnected.length`
- `n == isConnected[i].length`
- `isConnected[i][j] is 1 or 0.`
- `isConnected[i][i] == 1`
- `isConnected[i][j] == isConnected[j][i]`

Approach:

Pattern: Connected components

- **Represent:** Convert adjacency matrix to adjacency list for easier traversal.
- **Explore:** Loop through each node. If not visited, start a DFS/BFS and increment the province count.

Key idea

- A province is just a connected component. Every time the outer loop finds an unvisited city, a brand-new component starts → count++.
- One dfs(i) call marks the whole component, so later cities of the same province are skipped.

Common mistakes

- The matrix is symmetric, so pushing both adj[i] and adj[j] inside the double loop adds every edge twice — harmless for correctness, but remember it doubles the list size.
- Skip i == j (self-loop on the diagonal).

Graph leetcode problems

23 October 2026

Code:

number_of_provinces.cpp

```cpp
// DFS & BFS Solutions Included
class Solution {
public:
    // DFS Approach
    void dfs(int i, vector<vector<int>> &adjList, vector<bool> &visited) {
        visited[i] = true;
        for (auto it: adjList[i]) {
            if(!visited[it]) dfs(it,adjList,visited);
        }
    }

    int findCircleNum(vector<vector<int>>& isConnected) {
        int V = isConnected.size();
        vector<vector<int>> adjList(V);
        for(int i=0;i<V;i++){
            for(int j=0;j<V;j++) {
                if(isConnected[i][j]==1 && i!=j) {
                    adjList[i].push_back(j);
                    adjList[j].push_back(i);
                }
            }
        }
        vector<bool> visited(V, false);
        int count = 0;
        for(int i=0;i<V;i++) {
            if(!visited[i]) {
                count++;
                dfs(i,adjList,visited);
            }
        }
        return count;
    }
};
```

DRY-RUN:

Input: `isConnected = [[1,1,0,0,0],[1,1,0,0,0],[0,0,1,0,0],[0,0,0,1,1],[0,0,0,1,1]]`

Count provinces with DFS on the adjacency list

Step	What happens
1	Build adjList from the matrix (skip the diagonal). Nothing is visited yet, count = 0.
2	i = 0 is not visited → a new province starts, count = 1.
3	dfs(0): mark 0 visited. Neighbours: [1, 1].
4	dfs(1): mark 1 visited. Neighbours: [0, 0].
5	All neighbours of 1 are visited → return from dfs(1).
6	All neighbours of 0 are visited → return from dfs(0).
7	i = 1 was already visited inside an earlier DFS → skip.
8	i = 2 is not visited → a new province starts, count = 2.
9	dfs(2): mark 2 visited. Neighbours: [].
10	All neighbours of 2 are visited → return from dfs(2).

11	i = 3 is not visited → a new province starts, count = 3.
12	dfs(3): mark 3 visited. Neighbours: [4, 4].
13	dfs(4): mark 4 visited. Neighbours: [3, 3].
14	All neighbours of 4 are visited → return from dfs(4).
15	All neighbours of 3 are visited → return from dfs(3).
16	i = 4 was already visited inside an earlier DFS → skip.
17	Loop finished. Answer = 3 provinces: {0,1}, {2}, {3,4}.

Analysis:

- **Time Complexity:** $O(V^2)$
- **Space Complexity:** $O(V)$

Contest tips

- You can DFS directly on the matrix (row i is the neighbour list) and skip building adjList: still $O(V^2)$.
- DSU alternative: start with V components and decrement on every successful union.

NUMBER OF ISLANDS

Problem Statement:

200. Number of Islands

Medium | LeetCode | Open the full problem

You are given an `m x n` 2D grid `grid` of characters, where `'1'` represents land and `'0'` represents water. Return the number of islands.

An **island** is a group of land cells joined horizontally or vertically (not diagonally) and surrounded by water. You may assume that all four edges of the grid are surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```
Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j] is '0' or '1'.`

Approach:

Pattern: Connected components on a grid (flood fill)

- **Scan:** Iterate through every cell. If you find land ('1') that isn't visited, you found a new island.
- **Flood:** Use BFS to mark all connected land cells as visited so they aren't counted twice.

Key idea

- Each island is a connected component of land cells under 4-directional adjacency. The driver scans every cell; an unvisited '1' means a new island → count++ and one bfs() swallows the whole island.
- Total work is O(M·N): every cell is pushed into the queue at most once because visited is set at push time.

Common mistakes

- Mark visited when you push, not when you pop — otherwise the same cell can be enqueued by several neighbours (TLE / memory blow-up on big grids).

- The grid is char: compare with '1', not 1.
- Check bounds before indexing grid[nrow][ncol] (short-circuit order in the if matters).
- Diagonal cells are NOT adjacent here (only 4 directions). Some variants use 8-connectivity — just extend dr/dc.

Code:

number_of_islands.cpp

```cpp
class Solution {
public:
    void bfs(int m,int n, int i,int j,vector<vector<char>>& grid,vector<vector<bool>>& visited) {
        queue<pair<int,int>> q;
        visited[i][j] = true;
        q.push({i,j});
        int dr[4] = {-1, 1, 0, 0};
        int dc[4] = {0, 0, -1, 1};
        while(!q.empty()) {
            int row = q.front().first;
            int col = q.front().second;
            q.pop();
            for(int k=0;k<4;k++) {
                int nrow = row + dr[k], ncol = col + dc[k];
                if(nrow>=0 && ncol>=0 && nrow<m && ncol<n && grid[nrow][ncol]=='1' && !visited[nrow][ncol]) {
                    q.push({nrow,ncol});
                    visited[nrow][ncol] = true;
                }
            }
        }
    }
};
```

DRY-RUN:

Input: `grid = [["1","1","0","0","0"],["1","0","0","1","0"],["0","0","1","1","0"],["0","0","0","0","1"]]`

numIslands driver + your bfs(): one BFS per unvisited land cell

Step	What happens
1	Scan every cell with i, j. Water is skipped; an unvisited '1' starts a new island.
2	(0,0) is unvisited land → new island, count = 1. Call bfs(): mark it visited and push it.
3	Pop (0,0) and try 4 directions. Unvisited land found: down (1,0), right (0,1) → push & mark visited.
4	Pop (1,0) and try 4 directions. No unvisited land neighbour → nothing pushed.
5	Pop (0,1) and try 4 directions. No unvisited land neighbour → nothing pushed. Queue empty → island #1 fully explored.
6	(0,1) is land but visited[0][1] is already true (swallowed by an earlier BFS) → skip.
7	(1,0) is land but visited[1][0] is already true (swallowed by an earlier BFS) → skip.
8	(1,3) is unvisited land → new island, count = 2. Call bfs(): mark it visited and push it.
9	Pop (1,3) and try 4 directions. Unvisited land found: down (2,3) → push & mark visited.
10	Pop (2,3) and try 4 directions. Unvisited land found: left (2,2) → push & mark visited.

11	Pop (2,2) and try 4 directions. No unvisited land neighbour → nothing pushed. Queue empty → island #2 fully explored.
12	(2,2) is land but visited[2][2] is already true (swallowed by an earlier BFS) → skip.
13	(2,3) is land but visited[2][3] is already true (swallowed by an earlier BFS) → skip.
14	(3,4) is unvisited land → new island, count = 3. Call bfs(): mark it visited and push it.
15	Pop (3,4) and try 4 directions. No unvisited land neighbour → nothing pushed. Queue empty → island #3 fully explored.
16	Scan finished. Answer = 3 islands.

Analysis:

- **Time Complexity:** O(M × N)
- **Space Complexity:** O(M × N)

Contest tips

- You can drop the visited matrix by overwriting land with '0' (sink the island) — no extra grid besides the queue.
- Recursive DFS is shorter but can stack-overflow on a 300×300 all-land grid; BFS or an explicit stack is safer.
- Variants: Max Area of Island (LC 695 — return BFS size), Number of Enclaves (LC 1020 — BFS from the border first), Surrounded Regions (LC 130), Distinct Islands (LC 694 — record the shape).
- DSU also works: union each land cell with its right/down land neighbour, answer = #land − #successful unions.

FLOOD FILL

Problem Statement:

733. Flood Fill

Easy | LeetCode | Open the full problem

An image is represented by an `m x n` integer grid `image`, where `image[i][j]` is the color value of that pixel. You are also given three integers `sr`, `sc` and `color`. Perform a **flood fill** starting at pixel `image[sr][sc]`:

19. Start from the starting pixel and change its color to `color`.
20. Do the same for every pixel that is directly adjacent to it (up, down, left or right) and has the **same original color** as the starting pixel.
21. Keep repeating this for the adjacent pixels of each newly recolored pixel, as long as they share the starting pixel's original color.
22. Stop when there are no more adjacent pixels of the original color to update.

Return the modified image after the flood fill.

Example 1:

```
Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2
Output: [[2,2,2],[2,2,0],[2,0,1]]
Explanation: Starting from the center pixel (1, 1), every pixel connected to it by a path of the same original color (1) is recolored to 2. The bottom-right pixel is not recolored because it is not 4-directionally connected to the starting pixel.
```

Example 2:

```
Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0
Output: [[0,0,0],[0,0,0]]
Explanation: The starting pixel already has color 0, which equals the target color, so the image does not change.
```

Constraints:

- `m == image.length`
- `n == image[i].length`
- `1 <= m, n <= 50`
- `0 <= image[i][j], color < 2^16`
- `0 <= sr < m`
- `0 <= sc < n`

Approach:

Pattern: Flood fill (DFS on an implicit grid graph)

- Read initialColor = image[sr][sc] before painting, then DFS: a pixel joins the fill iff it is in bounds, not yet visited and still has initialColor.
- All the checks sit at the top of dfs() (“call first, check inside”), so each cell is reached from up to 4 neighbours but painted once → O(M·N).

Common mistakes

- If color == initialColor and you have no visited array, the "same colour" test never fails → infinite recursion. Either keep visited (as you do) or return early when the colours are equal.

- Capture initialColor before the first write; comparing against image[sr][sc] later reads the already-painted value.
- Bounds must be tested before image[r][c] is read — your if does this thanks to short-circuit ||.

Code:

flood_fill.cpp

```cpp
class Solution {
public:
    void dfs(int r, int c, vector<vector<int>>& image, vector<vector<bool>>& visited, int m, int n, int initialColor, int newColor) {
        if(r < 0 || c < 0 || r >= m || c >= n || visited[r][c] || image[r][c] != initialColor) return;
        visited[r][c]=true;
        image[r][c]=newColor;
        dfs(r-1,c,image,visited,m,n,initialColor,newColor);
        dfs(r+1,c,image,visited,m,n,initialColor,newColor);
        dfs(r,c-1,image,visited,m,n,initialColor,newColor);
        dfs(r,c+1,image,visited,m,n,initialColor,newColor);
    }
};
```

DRY-RUN:

Input: `image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2`

floodFill driver + your recursive dfs() (up, down, left, right)

Step	What happens
1	initialColor = image[1][1] = 1, newColor = 2. Start with dfs(1,1).
2	dfs(1,1): in bounds, unvisited and colour 1 → set visited[1][1], paint 2. Now recurse up, down, left, right.
3	dfs(0,1): in bounds, unvisited and colour 1 → set visited[0][1], paint 2. Now recurse up, down, left, right.
4	From (0,1) go up: dfs(-1,1) returns immediately — out of bounds.
5	From (0,1) go down: dfs(1,1) returns immediately — visited[1][1] is already true.
6	dfs(0,0): in bounds, unvisited and colour 1 → set visited[0][0], paint 2. Now recurse up, down, left, right.
7	From (0,0) go up: dfs(-1,0) returns immediately — out of bounds.
8	dfs(1,0): in bounds, unvisited and colour 1 → set visited[1][0], paint 2. Now recurse up, down, left, right.
9	From (1,0) go up: dfs(0,0) returns immediately — visited[0][0] is already true.
10	dfs(2,0): in bounds, unvisited and colour 1 → set visited[2][0], paint 2. Now recurse up, down, left, right.
11	From (2,0) go up: dfs(1,0) returns immediately — visited[1][0] is already true.
12	From (2,0) go down: dfs(3,0) returns immediately — out of bounds.
13	From (2,0) go left: dfs(2,-1) returns immediately — out of bounds.
14	From (2,0) go right: dfs(2,1) returns immediately — image[2][1] = 0 ≠ initialColor.
15	From (1,0) go left: dfs(1,-1) returns immediately — out of bounds.
16	From (1,0) go right: dfs(1,1) returns immediately — visited[1][1] is already true.
17	From (0,0) go left: dfs(0,-1) returns immediately — out of bounds.

18	From (0,0) go right: dfs(0,1) returns immediately — visited[0][1] is already true.
19	dfs(0,2): in bounds, unvisited and colour 1 → set visited[0][2], paint 2. Now recurse up, down, left, right.
20	From (0,2) go up: dfs(-1,2) returns immediately — out of bounds.
21	From (0,2) go down: dfs(1,2) returns immediately — image[1][2] = 0 ≠ initialColor.
22	From (0,2) go left: dfs(0,1) returns immediately — visited[0][1] is already true.
23	From (0,2) go right: dfs(0,3) returns immediately — out of bounds.
24	From (1,1) go down: dfs(2,1) returns immediately — image[2][1] = 0 ≠ initialColor.
25	From (1,1) go left: dfs(1,0) returns immediately — visited[1][0] is already true.
26	From (1,1) go right: dfs(1,2) returns immediately — image[1][2] = 0 ≠ initialColor.
27	All calls returned. Final image = [[2,2,2],[2,2,0],[2,0,1]]. The bottom-right 1 is not 4-connected to the start, so it stays 1.

Analysis:

- **Time Complexity:** O(M × N)
- **Space Complexity:** O(M × N)

Contest tips

- With the early-return trick you can skip visited entirely: painted pixels no longer equal initialColor.
- Recursion depth can reach M·N (snake-shaped region); for huge grids switch to BFS / an explicit stack.
- Same template: Number of Islands (LC 200), Surrounded Regions (LC 130), Pacific Atlantic (LC 417), paint-bucket tools.

ROTTEN ORANGES

Problem Statement:

994. Rotting Oranges

Medium | LeetCode | Open the full problem

You are given an `m x n` grid in which each cell holds one of three values:

- `0`: an empty cell,
- `1`: a fresh orange,
- `2`: a rotten orange.

Every minute, each fresh orange that is 4-directionally adjacent (up, down, left, right) to a rotten orange becomes rotten.

Return the minimum number of minutes that must pass until no cell contains a fresh orange. If this can never happen, return `-1`.

Example 1:

```
Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4
```

Example 2:

```
Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom-left corner (row 2, column 0) never rots, because rotting only spreads 4-directionally.
```

Example 3:

```
Input: grid = [[0,2]]
Output: 0
Explanation: There are no fresh oranges at minute 0, so the answer is 0.
```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 10`
- `grid[i][j] is 0, 1, or 2.`

Approach:

Pattern: Multi-source BFS (level = minute)

- All initially rotten oranges spread simultaneously, so push them all at t = 0 before the loop. BFS then pops cells in non-decreasing t, and the first time a fresh orange is reached is the earliest minute it rots.
- The answer is the largest t ever popped (your time = max(t, time)), provided no fresh orange is left.

Common mistakes

- Running a separate BFS from each rotten orange gives wrong times (and $O((MN)^2)$). One queue, all sources.
- Return -1 only if a fresh orange remains; a grid with no fresh oranges (even no oranges at all) answers 0.

- Mark rotten when pushing, otherwise two rotten neighbours enqueue the same orange twice.

Code:

rotten_oranges.cpp

```cpp
class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        int m=grid.size(); int n = grid[0].size();
        queue<pair<pair<int,int>,int>> q;
        vector<vector<int>> visited(m,vector<int>(n,0));
        for(int i=0;i<m;i++) {
            for(int j=0;j<n;j++) {
                if(grid[i][j]==2) { q.push({{i,j},0}); visited[i][j]=2; }
                else if(grid[i][j]==1) visited[i][j]=1;
            }
        }
        int time=0;
        int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
        while(!q.empty()){
            int r=q.front().first.first; int c=q.front().first.second; int
t=q.front().second;
            q.pop(); time=max(t,time);
            for(int k=0;k<4;k++){
                int nr=r+dr[k], nc=c+dc[k];
                if(nr>=0 && nr<m && nc>=0 && nc<n && visited[nr][nc]==1 &&
grid[nr][nc]==1){
                    q.push({{nr,nc},t+1}); visited[nr][nc]=2;
                }
            }
        }
        for(int i=0;i<m;i++) for(int j=0;j<n;j++) if(visited[i][j]==1) return -1;
        return time;
    }
};
```

DRY-RUN:

Input: `grid = [[2,1,1],[1,1,0],[0,1,1]]`

Multi-source BFS — all rotten oranges start in the queue at t = 0

Step	What happens
1	Initial scan: every rotten orange is pushed with t = 0 (multi-source BFS) and gets visited = 2; fresh ones get visited = 1.
2	Pop (0,0) with t = 0; time = max(0, 0) = 0. Fresh neighbours (1,0), (0,1) rot → push with t = 1.
3	Pop (1,0) with t = 1; time = max(1, 0) = 1. Fresh neighbours (1,1) rot → push with t = 2.
4	Pop (0,1) with t = 1; time = max(1, 1) = 1. Fresh neighbours (0,2) rot → push with t = 2.
5	Pop (1,1) with t = 2; time = max(2, 1) = 2. Fresh neighbours (2,1) rot → push with t = 3.
6	Pop (0,2) with t = 2; time = max(2, 2) = 2. No fresh neighbour.
7	Pop (2,1) with t = 3; time = max(3, 2) = 3. Fresh neighbours (2,2) rot → push with t = 4.
8	Pop (2,2) with t = 4; time = max(4, 3) = 4. No fresh neighbour.
9	Queue empty and no visited[i][j] == 1 left → every orange rotted. Answer = 4 minutes.

Analysis:

- **Time Complexity:** O(M × N)
- **Space Complexity:** O(M × N)

Contest tips

- Level-by-level alternative: count fresh oranges, process the queue in rounds of q.size(), minutes++ per round that rots something, decrement fresh on each rot; answer fresh == 0 ? minutes : -1.
- Same pattern: 01 Matrix (LC 542), Walls and Gates (LC 286), As Far from Land as Possible (LC 1162), Shortest Bridge (LC 934, BFS from a whole island).

01 MATRIX

Problem Statement:

542. 01 Matrix

Medium | LeetCode | Open the full problem

Given an `m x n` binary matrix `mat`, return a matrix of the same size where each cell holds the distance from that cell to the nearest `0` in `mat`.

The distance between two cells that share an edge (up, down, left, right) is `1`.

Example 1:

```
Input: mat = [[0,0,0],[0,1,0],[0,0,0]]
Output: [[0,0,0],[0,1,0],[0,0,0]]
```

Example 2:

```
Input: mat = [[0,0,0],[0,1,0],[1,1,1]]
Output: [[0,0,0],[0,1,0],[1,2,1]]
```

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 10^4`
- `1 <= m * n <= 10^4`
- `mat[i][j] is either 0 or 1.`
- `There is at least one 0 in mat.`

Approach:

Pattern: Multi-source BFS (reverse the question)

- Instead of a BFS from every 1 to its nearest 0 ($O((MN)^2)$), start from all zeros at once with distance 0. The first time BFS reaches a cell is its distance to the closest zero.
- Each cell is pushed once (visited at push time), so the whole thing is $O(M \cdot N)$.

Common mistakes

- Do not BFS separately from each 1 — TLE on 10^4 cells.
- Mark visited on push, not on pop, or a cell gets enqueued once per neighbour.
- Your code sets dist[r][c] = s on pop; setting dist[nr][nc] = s + 1 on push is equivalent and lets you drop s from the queue.

Code:

01_matrix.cpp

```cpp
class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
        int m=mat.size(); int n=mat[0].size();
        vector<vector<int>> visited(m,vector<int>(n,0));
        queue<pair<pair<int,int> ,int>> q;
        vector<vector<int>> dist(m,vector<int>(n,0));
        for(int i=0;i<m;i++){
            for(int j=0;j<n;j++){
```

```
                    if(mat[i][j]==0){ visited[i][j]=1; q.push({{i,j},0}); }
                }
            }
            int dr[]={-1,1,0,0}, dc[]={0,0,-1,1};
            while(!q.empty()){
                int r=q.front().first.first; int c=q.front().first.second; int
s=q.front().second;
                q.pop(); dist[r][c]=s;
                for(int k=0;k<4;k++){
                    int nr=r+dr[k], nc=c+dc[k];
                    if(nr>=0 && nr<m && nc>=0 && nc<n && !visited[nr][nc]){
                        q.push({{nr,nc},s+1}); visited[nr][nc]=1;
                    }
                }
            }
            return dist;
        }
    };
```

DRY-RUN:

Input: `mat = [[0,0,0],[0,1,0],[1,1,1]]`

Multi-source BFS from every 0 — dist[r][c] is written when (r,c) is popped

Step	What happens
1	Push every 0 with s = 0 and mark it visited — 5 sources start the BFS together.
2	Pop (0,0), set dist[0][0] = 0. All neighbours already visited.
3	Pop (0,1), set dist[0][1] = 0. Unvisited neighbours (1,1) → push with s = 1.
4	Pop (0,2), set dist[0][2] = 0. All neighbours already visited.
5	Pop (1,0), set dist[1][0] = 0. Unvisited neighbours (2,0) → push with s = 1.
6	Pop (1,2), set dist[1][2] = 0. Unvisited neighbours (2,2) → push with s = 1.
7	Pop (1,1), set dist[1][1] = 1. Unvisited neighbours (2,1) → push with s = 2.
8	Pop (2,0), set dist[2][0] = 1. All neighbours already visited.
9	Pop (2,2), set dist[2][2] = 1. All neighbours already visited.
10	Pop (2,1), set dist[2][1] = 2. All neighbours already visited.
11	Queue empty. Answer = [[0,0,0],[0,1,0],[1,2,1]].

Analysis:

- **Time Complexity:** O(M × N)
- **Space Complexity:** O(M × N)

Contest tips

- DP alternative with two sweeps: top-left → bottom-right using up/left, then bottom-right → top-left using down/right. Same O(M·N), no queue.
- Initialise dist with −1 for 1-cells and use dist == -1 as "unvisited" to drop the separate visited grid.
- Related: Rotting Oranges (LC 994), Map of Highest Peak (LC 1765 — essentially this problem), As Far from Land as Possible (LC 1162).

BIPARTITE GRAPH

Problem Statement:

785. Is Graph Bipartite?

Medium | LeetCode | Open the full problem

There is an **undirected** graph with `n` nodes numbered `0` to `n - 1`. It is given as a 2D array `graph`, where `graph[u]` lists the nodes adjacent to node `u`: for every `v` in `graph[u]`, there is an undirected edge between `u` and `v`. The graph has these properties:

- No self-edges: `graph[u]` does not contain `u`.
- No parallel edges: `graph[u]` has no duplicate values.
- If `v` is in `graph[u]`, then `u` is in `graph[v]` (the graph is undirected).
- The graph may be disconnected: there may be pairs of nodes `u` and `v` with no path between them.

A graph is **bipartite** if its nodes can be split into two independent sets `A` and `B` such that **every** edge connects a node in `A` with a node in `B`.

Return `true` if and only if the graph is bipartite.

Example 1:

```
Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]
Output: false
Explanation: There is no way to split the nodes into two independent sets such that every edge connects a node in one set with a node in the other.
```

Example 2:

```
Input: graph = [[1,3],[0,2],[1,3],[0,2]]
Output: true
Explanation: The nodes can be split into two sets: {0, 2} and {1, 3}.
```

Constraints:

- `graph.length == n`
- `1 <= n <= 100`
- `0 <= graph[u].length < n`
- `0 <= graph[u][i] <= n - 1`
- `graph[u] does not contain u.`
- `All the values of graph[u] are unique.`
- `If graph[u] contains v, then graph[v] contains u.`

Approach:

Pattern: 2-colouring / odd-cycle detection

- A graph is bipartite ⇔ it has no odd-length cycle ⇔ it can be 2-coloured. Colour a node c, force every neighbour to !c; any neighbour already holding c is a contradiction.
- Inside a connected component the colouring is forced once the first node is chosen, so a greedy DFS/BFS is exact. O(V + E).

Common mistakes

- The graph can be disconnected — loop over all nodes and start a DFS from every uncoloured one (your outer loop does this). Checking only node 0 is the classic WA.

- Propagate the result of the recursive call (if(!dfs(...)) return false;); calling dfs and ignoring its return value loses the conflict.
- Use a 3-state array (−1 / 0 / 1); a plain visited bool cannot tell the two sides apart.

Code:

bipartite_graph.cpp

```cpp
class Solution {
public:
    bool dfs(int node,vector<vector<int>> &adj,vector<int> &color,int c) {
        color[node] = c;
        for(auto it: adj[node]) {
            if(color[it]==-1) { if(!dfs(it,adj,color,!c)) return false; }
            else if(color[it]==c) return false;
        }
        return true;
    }
    bool isBipartite(vector<vector<int>>& graph) {
        int n=graph.size();
        vector<int> color(n,-1);
        for(int i=0;i<n;i++) {
            if(color[i]==-1) { if(!dfs(i,graph,color,0)) return false; }
        }
        return true;
    }
};
```

DRY-RUN:

Input: `graph = [[1,3],[0,2],[1,3],[0,2],[5],[4]]`

DFS 2-colouring with color[] (0 / 1, -1 = uncoloured)

Step	What happens
1	All color[i] = -1 (uncoloured). The outer loop starts dfs(i, adj, color, 0) from every uncoloured node, so disconnected parts are handled too.
2	dfs(0, c=0): set color[0] = 0. Every neighbour must end up with colour 1.
3	Neighbour 1 is uncoloured → recurse with !c = 1.
4	dfs(1, c=1): set color[1] = 1. Every neighbour must end up with colour 0.
5	Neighbour 0 already has the opposite colour 0 → consistent, continue.
6	Neighbour 2 is uncoloured → recurse with !c = 0.
7	dfs(2, c=0): set color[2] = 0. Every neighbour must end up with colour 1.
8	Neighbour 1 already has the opposite colour 1 → consistent, continue.
9	Neighbour 3 is uncoloured → recurse with !c = 1.
10	dfs(3, c=1): set color[3] = 1. Every neighbour must end up with colour 0.
11	Neighbour 0 already has the opposite colour 0 → consistent, continue.
12	Neighbour 2 already has the opposite colour 0 → consistent, continue.
13	Back in dfs(2): the subtree under 3 coloured fine.
14	Back in dfs(1): the subtree under 2 coloured fine.

15	Back in dfs(0): the subtree under 1 coloured fine.
16	Neighbour 3 already has the opposite colour 1 → consistent, continue.
17	Outer loop: i = 1 already coloured → skip.
18	Outer loop: i = 2 already coloured → skip.
19	Outer loop: i = 3 already coloured → skip.
20	Outer loop: i = 4 is still uncoloured → new component, start dfs(4, c=0).
21	dfs(4, c=0): set color[4] = 0. Every neighbour must end up with colour 1.
22	Neighbour 5 is uncoloured → recurse with !c = 1.
23	dfs(5, c=1): set color[5] = 1. Every neighbour must end up with colour 0.
24	Neighbour 4 already has the opposite colour 0 → consistent, continue.
25	Back in dfs(4): the subtree under 5 coloured fine.
26	Outer loop: i = 5 already coloured → skip.
27	Every node coloured without a conflict → the two colours are the two sides. Answer = true.

Analysis:

- **Time Complexity:** O(V + E)
- **Space Complexity:** O(V)

Contest tips

- BFS version avoids deep recursion: pop u, for each v uncoloured → color[v] = 1 - color[u]; same colour → false.
- DSU version: union all neighbours of u together; if find(u) == find(v) for some neighbour v → not bipartite.
- Related: Possible Bipartition (LC 886, build the graph from dislikes), bipartite matching, "does the graph contain an odd cycle?".

EVENTUAL SAFE NODES

Problem Statement:

802. Find Eventual Safe States

Medium | LeetCode | Open the full problem

There is a directed graph with `n` nodes labeled `0` to `n - 1`. It is given as a 0-indexed 2D integer array `graph`, where `graph[i]` is an integer array of the nodes that node `i` has an outgoing edge to (there is an edge from `i` to each node in `graph[i]`).

A node is a **terminal node** if it has no outgoing edges. A node is a **safe node** if every possible path that starts from it leads to a terminal node (or to another safe node).

Return an array containing all the safe nodes of the graph, sorted in **ascending** order.

Example 1:

```
Input: graph = [[1,2],[2,3],[5],[0],[5],[],[]]
Output: [2,4,5,6]
Explanation: Nodes 5 and 6 are terminal nodes because they have no outgoing edges. Every path starting at nodes 2, 4, 5 or 6 ends at node 5 or node 6.
```

Example 2:

```
Input: graph = [[1,2,3,4],[1,2],[3,4],[0,4],[]]
Output: [4]
Explanation: Only node 4 is a terminal node, and every path starting at node 4 leads to node 4.
```

Constraints:

- `n == graph.length`
- `1 <= n <= 10^4`
- `0 <= graph[i].length <= n`
- `0 <= graph[i][j] <= n - 1`
- `graph[i] is sorted in a strictly increasing order.`
- `The graph may contain self-loops.`
- `The number of edges in the graph will be in the range [1, 4 * 10^4].`

Approach:

Pattern: Cycle detection in a directed graph (3-state DFS)

- A node is unsafe iff some path from it reaches a cycle. DFS with a "path" array: a neighbour with pVis = 1 closes a cycle.
- When a cycle is found, dfs returns true immediately without resetting pVis or setting check — every node on that path stays marked, and any later node that reaches them sees pVis = 1 and is unsafe too. Nodes whose DFS finishes cleanly get check = 1.
- **O(V + E):** each node is expanded once.

Common mistakes

- Leaving pVis[node] = 1 on the "cycle found" path is intentional here — resetting it would let later DFSs treat those nodes as safe.
- Using only vis[] (undirected-style) misreports edges into already-finished nodes as cycles.

- The answer must be sorted ascending — collecting with for i in 0..V-1 if(check[i]) gives that for free.

Code:

eventual_safe_nodes.cpp

```cpp
class Solution {
public:
    bool dfs(int node,vector<vector<int>> &adj,vector<int> &vis, vector<int> &pVis, vector<int> &check) {
        vis[node]=1; pVis[node]=1;
        for(auto it: adj[node]) {
            if(!vis[it]) { if(dfs(it,adj,vis,pVis,check)) return true; }
            else if(pVis[it]) return true;
        }
        check[node]=1; pVis[node]=0;
        return false;
    }
    vector<int> eventualSafeNodes(vector<vector<int>>& graph) {
        int V = graph.size();
        vector<int> vis(V,0), pVis(V,0), check(V,0), safe;
        for(int i=0;i<V;i++) if(!vis[i]) dfs(i,graph,vis,pVis,check);
        for(int i=0;i<V;i++) if(check[i]) safe.push_back(i);
        return safe;
    }
};
```

DRY-RUN:

Input: `graph = [[1,2],[2,3],[5],[0],[5],[],[]]`

DFS with vis / pVis / check — a node is safe iff its DFS finishes without hitting the path

Step	What happens
1	Run dfs from every unvisited node. pVis marks the current recursion path; check marks proven-safe nodes. Start with dfs(0).
2	dfs(0): vis[0] = pVis[0] = 1 (now on the current path). Out-edges → [1, 2].
3	1 unvisited → recurse dfs(1).
4	dfs(1): vis[1] = pVis[1] = 1 (now on the current path). Out-edges → [2, 3].
5	2 unvisited → recurse dfs(2).
6	dfs(2): vis[2] = pVis[2] = 1 (now on the current path). Out-edges → [5].
7	5 unvisited → recurse dfs(5).
8	dfs(5): vis[5] = pVis[5] = 1 (now on the current path). Out-edges → [none].
9	All out-edges of 5 handled without a cycle → check[5] = 1, pVis[5] = 0 (leave the path), return false.
10	Back in dfs(2): no cycle through 5.
11	All out-edges of 2 handled without a cycle → check[2] = 1, pVis[2] = 0 (leave the path), return false.
12	Back in dfs(1): no cycle through 2.
13	3 unvisited → recurse dfs(3).
14	dfs(3): vis[3] = pVis[3] = 1 (now on the current path). Out-edges → [0].
15	0 is visited and pVis[0] = 1 → edge 3→0 points back into the current path: cycle! return true.

16	dfs(3) returned true (cycle below) → dfs(1) returns true immediately, so check[1] stays 0 and pVis[1] stays 1.
17	dfs(1) returned true (cycle below) → dfs(0) returns true immediately, so check[0] stays 0 and pVis[0] stays 1.
18	Outer loop: i = 1 already visited → skip.
19	Outer loop: i = 2 already visited → skip.
20	Outer loop: i = 3 already visited → skip.
21	Outer loop: i = 4 not visited → dfs(4).
22	dfs(4): vis[4] = pVis[4] = 1 (now on the current path). Out-edges → [5].
23	5 is visited but pVis[5] = 0 (not on the current path) → already explored with no cycle, skip.
24	All out-edges of 4 handled without a cycle → check[4] = 1, pVis[4] = 0 (leave the path), return false.
25	Outer loop: i = 5 already visited → skip.
26	Outer loop: i = 6 not visited → dfs(6).
27	dfs(6): vis[6] = pVis[6] = 1 (now on the current path). Out-edges → [none].
28	All out-edges of 6 handled without a cycle → check[6] = 1, pVis[6] = 0 (leave the path), return false.
29	Collect every i with check[i] = 1: Answer = [2,4,5,6]. Nodes 0, 1, 3 lie on the cycle 0→1→3→0.

Analysis:

- **Time Complexity:** O(V + E)
- **Space Complexity:** O(V)

Contest tips

- BFS alternative (Kahn on the reversed graph): reverse all edges, start from terminal nodes (out-degree 0) and peel nodes whose out-degree drops to 0 — those are exactly the safe nodes.
- The three arrays can merge into one state[]: 0 = unvisited, 1 = on path / unsafe, 2 = safe.
- Related: Course Schedule I/II (LC 207/210) use the same vis + pVis DFS.

COURSE SCHEDULE

Problem Statement:

207. Course Schedule

Medium | LeetCode | Open the full problem

You must take `numCourses` courses in total, labeled `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a_i, b_i]` means you have to take course `b_i` before you can take course `a_i`.

For example, the pair `[0, 1]` means that course `1` must be completed before course `0`.

Return `true` if it is possible to finish all the courses. Otherwise, return `false`.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are 2 courses. To take course 1 you must first finish course 0, which is possible.
```

Example 2:

```
Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: There are 2 courses. Course 1 requires course 0, and course 0 also requires course 1. This is a cycle, so it is impossible.
```

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- `All the pairs prerequisites[i] are unique.`

Approach:

Pattern: Cycle detection in a directed graph / topological sort

- All courses can be finished ⇔ the prerequisite graph has no directed cycle (it is a DAG).
- In DFS, an edge to a node that is still on the recursion path (pVis = 1) is a back edge = cycle. An edge to a visited node that has already left the path is a cross/forward edge — harmless.

Common mistakes

- Using only vis[] (the undirected trick) is wrong: in the diamond 0→1→3, 0→2→3 node 3 is reached twice without any cycle.
- Forgetting pVis[node] = 0 on the way out turns every cross edge into a fake cycle.
- Edge direction: [a,b] means b → a. Flipping it does not change the yes/no answer here, but it matters for Course Schedule II (the order).
- Courses with no prerequisites still need the outer loop over all numCourses.

Code:

course_schedule.cpp

```cpp
class Solution {
public:
    bool dfs(int node,vector<vector<int>> &adj,vector<int> &vis, vector<int> &pVis) {
        vis[node]=1; pVis[node]=1;
        for(auto it: adj[node]) {
            if(!vis[it]) { if(dfs(it,adj,vis,pVis)) return true; }
            else if(pVis[it]) return true;
        }
        pVis[node]=0; return false;
    }
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> adj(numCourses);
        for(auto it:prerequisites) adj[it[1]].push_back(it[0]);
        vector<int> vis(numCourses,0), pVis(numCourses,0);
        for(int i=0;i<numCourses;i++) {
            if(!vis[i]) if(dfs(i,adj,vis,pVis)) return false;
        }
        return true;
    }
};
```

DRY-RUN:

Input: `numCourses = 5, prerequisites = [[1,0],[2,0],[3,1],[3,2],[4,3]]`

Directed-cycle detection with vis[] + pVis[] (path) DFS

Step	What happens
1	Build adj: for each [a,b] push a into adj[b] (take b before a). Then search for a directed cycle, starting with dfs(0).
2	dfs(0): vis[0] = pVis[0] = 1 (now on the current path). Out-edges → [1, 2].
3	1 unvisited → recurse dfs(1).
4	dfs(1): vis[1] = pVis[1] = 1 (now on the current path). Out-edges → [3].
5	3 unvisited → recurse dfs(3).
6	dfs(3): vis[3] = pVis[3] = 1 (now on the current path). Out-edges → [4].
7	4 unvisited → recurse dfs(4).
8	dfs(4): vis[4] = pVis[4] = 1 (now on the current path). Out-edges → [none].
9	All out-edges of 4 handled without a cycle → pVis[4] = 0 (leave the path), return false.
10	Back in dfs(3): no cycle through 4.
11	All out-edges of 3 handled without a cycle → pVis[3] = 0 (leave the path), return false.
12	Back in dfs(1): no cycle through 3.
13	All out-edges of 1 handled without a cycle → pVis[1] = 0 (leave the path), return false.
14	Back in dfs(0): no cycle through 1.
15	2 unvisited → recurse dfs(2).
16	dfs(2): vis[2] = pVis[2] = 1 (now on the current path). Out-edges → [3].
17	3 is visited but pVis[3] = 0 (not on the current path) → already explored with no cycle, skip.
18	All out-edges of 2 handled without a cycle → pVis[2] = 0 (leave the path), return false.

19	Back in dfs(0): no cycle through 2.
20	All out-edges of 0 handled without a cycle → pVis[0] = 0 (leave the path), return false.
21	Outer loop: i = 1 already visited → skip.
22	Outer loop: i = 2 already visited → skip.
23	Outer loop: i = 3 already visited → skip.
24	Outer loop: i = 4 already visited → skip.
25	Every DFS finished without a back edge → the prerequisite graph is a DAG. Answer = true.

Analysis:

- **Time Complexity:** O(V + E)
- **Space Complexity:** O(V + E)

Contest tips

- Kahn's BFS: push all in-degree-0 nodes, pop & decrement; if fewer than numCourses nodes get popped there is a cycle. Iterative (no recursion depth issues) and directly yields the order for LC 210.
- A single state[] array (0 unvisited, 1 visiting, 2 done) replaces vis + pVis.
- Related: Course Schedule II (LC 210), Alien Dictionary (LC 269), Parallel Courses (LC 1136 — longest path in a DAG), Eventual Safe States (LC 802).

NETWORK DELAY TIME

Problem Statement:

743. Network Delay Time

Medium | LeetCode | Open the full problem

You are given a network of `n` nodes labeled `1` to `n`, and a list `times` of travel times along **directed** edges, where `times[i] = (uᵢ, vᵢ, wᵢ)`: `uᵢ` is the source node, `vᵢ` is the target node, and `wᵢ` is the time it takes a signal to travel from source to target.

A signal is sent from a given node `k`. Return the **minimum** time it takes for all `n` nodes to receive the signal. If some node can never receive it, return `-1`.

Example 1:

```
Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
Output: 2
```

Example 2:

```
Input: times = [[1,2,1]], n = 2, k = 1
Output: 1
```

Example 3:

```
Input: times = [[1,2,1]], n = 2, k = 2
Output: -1
```

Constraints:

- `1 <= k <= n <= 100`
- `1 <= times.length <= 6000`
- `times[i].length == 3`
- `1 <= ui, vi <= n`
- `ui != vi`
- `0 <= wi <= 100`
- `All the pairs (ui, vi) are unique. (i.e., no multiple edges.)`

Approach:

Pattern: Single-source shortest path (Dijkstra)

- The signal reaches node v at time dist[v] = shortest path from k. Everyone has heard it once the slowest node has → answer = max(dist[i]).
- Edge weights are non-negative, so the smallest {d, node} in the min-heap is always final — that greedy fact is all Dijkstra is.
- Any node still at 1e9 was never reached → return -1.

Common mistakes

- Nodes are 1-indexed: size arrays n+1 and loop i = 1..n (index 0 is unused and must not be counted).
- This code has no if (d > dist[node]) continue;. It is still correct (a stale entry cannot improve anything), but it re-scans adjacency lists. Adding the check is the standard optimisation.
- The heap needs greater<> — the default priority_queue is a max-heap, which silently gives wrong/slow results.

- Edges are directed here: only adj[u].push_back({v,w}).

Code:

network_delay_time.cpp

```cpp
class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        vector<vector<pair<int,int>>> adj(n+1);
        for(auto it:times) adj[it[0]].push_back({it[1],it[2]});
        priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
        vector<int> dist(n+1, 1e9);
        dist[k]=0; pq.push({0,k});
        while(!pq.empty()){
            int d = pq.top().first; int node = pq.top().second; pq.pop();
            for(auto it: adj[node]){
                if(d + it.second < dist[it.first]){
                    dist[it.first] = d + it.second; pq.push({dist[it.first], it.first});
                }
            }
        }
        int ans=0;
        for(int i=1;i<=n;i++) {
            if(dist[i]==1e9) return -1;
            ans=max(ans, dist[i]);
        }
        return ans;
    }
};
```

DRY-RUN:

Input: `times = [[1,2,2],[1,3,5],[2,3,1],[2,4,6],[3,4,2],[4,5,1],[3,5,7]], n = 5, k = 1`

Dijkstra from k with a min-heap of {dist, node}

Step	What happens
1	Build adj (directed). dist[k=1] = 0, everything else 1e9 (∞). Push {0, 1}.
2	Pop {0, 1} → d = 0, node = 1. Relax its 2 outgoing edge(s).
3	Edge 1→2 (w=2): d + w = 2 < dist[2] = ∞ → update dist[2] = 2 and push {2, 2}.
4	Edge 1→3 (w=5): d + w = 5 < dist[3] = ∞ → update dist[3] = 5 and push {5, 3}.
5	Pop {2, 2} → d = 2, node = 2. Relax its 2 outgoing edge(s).
6	Edge 2→3 (w=1): d + w = 3 < dist[3] = 5 → update dist[3] = 3 and push {3, 3}.
7	Edge 2→4 (w=6): d + w = 8 < dist[4] = ∞ → update dist[4] = 8 and push {8, 4}.
8	Pop {3, 3} → d = 3, node = 3. Relax its 2 outgoing edge(s).
9	Edge 3→4 (w=2): d + w = 5 < dist[4] = 8 → update dist[4] = 5 and push {5, 4}.
10	Edge 3→5 (w=7): d + w = 10 < dist[5] = ∞ → update dist[5] = 10 and push {10, 5}.
11	Pop {5, 3} → d = 5, node = 3. Stale entry (dist[3] is already 3); the code has no skip-check, so it still scans the neighbours — nothing will improve.
12	Edge 3→4 (w=2): 5 + 2 = 7 is not < dist[4] = 5 → no change.
13	Edge 3→5 (w=7): 5 + 7 = 12 is not < dist[5] = 10 → no change.

14	Pop {5, 4} → d = 5, node = 4. Relax its 1 outgoing edge(s).
15	Edge 4→5 (w=1): d + w = 6 < dist[5] = 10 → update dist[5] = 6 and push {6, 5}.
16	Pop {6, 5} → d = 6, node = 5. No outgoing edges.
17	Pop {8, 4} → d = 8, node = 4. Stale entry (dist[4] is already 5); the code has no skip-check, so it still scans the neighbours — nothing will improve.
18	Edge 4→5 (w=1): 8 + 1 = 9 is not < dist[5] = 6 → no change.
19	Pop {10, 5} → d = 10, node = 5. Stale entry (dist[5] is already 6); the code has no skip-check, so it still scans the neighbours — nothing will improve.
20	Heap empty. No dist[i] is 1e9, so every node got the signal. Answer = max(dist) = 6 (node 5 hears it last).

Analysis:

- **Time Complexity:** O(E log V)
- **Space Complexity:** O(V + E)

Contest tips

- Complexity O(E log E): every successful relaxation pushes once.
- n ≤ 100 here, so Bellman–Ford (O(nE)) or even Floyd–Warshall would also pass — pick the one you can type bug-free fastest.
- Related: 1514 Path with Max Probability (max-heap), 1631 Path with Minimum Effort (dist = max edge), 787 Cheapest Flights (extra state).

CHEAPEST FLIGHTS WITHIN K STOPS

Problem Statement:

787. Cheapest Flights Within K Stops

Medium | LeetCode | Open the full problem

There are `n` cities connected by some number of flights. You are given an array `flights` where `flights[i] = [from_i, to_i, price_i]` means there is a one-way flight from city `from_i` to city `to_i` that costs `price_i`.

You are also given three integers `src`, `dst` and `k`. Return the **cheapest price** to travel from `src` to `dst` using **at most** `k` stops (intermediate cities). If no such route exists, return `-1`.

Example 1:

```
Input: n = 4, flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src = 0, dst = 3, k = 1
Output: 700
Explanation: The optimal route is 0 -> 1 -> 3 with cost 100 + 600 = 700. The route 0 -> 1 -> 2 -> 3 is cheaper, but it uses 2 stops, which exceeds k = 1, so it is not allowed.
```

Example 2:

```
Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 1
Output: 200
Explanation: The cheapest route with at most 1 stop is 0 -> 1 -> 2, costing 100 + 100 = 200.
```

Example 3:

```
Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 0
Output: 500
Explanation: With no stops allowed, the only option is the direct flight 0 -> 2, costing 500.
```

Constraints:

- `1 <= n <= 100`
- `0 <= flights.length <= (n * (n - 1) / 2)`
- `flights[i].length == 3`
- `0 <= fromi, toi < n`
- `fromi != toi`
- `1 <= pricei <= 10^4`
- `There will not be any multiple flights between two cities.`
- `0 <= src, dst, k < n`
- `src != dst`

Approach:

Pattern: Shortest path with a resource limit (state = node + stops)

- **Plain Dijkstra is wrong here:** the cheapest way to reach a node may use too many stops, and a pricier-but-shorter route can be the only one that still reaches dst in time.
- **Fix:** make the state (node, stops). dist[v][s] = cheapest cost to stand at v after exactly s flights. Now states never compete with each other unfairly.
- Because the heap pops by cost, the first time dst is popped its cost is optimal.

Common mistakes

- "k stops" = k + 1 edges. Off-by-one on stops > k vs stops >= k is the #1 bug.
- Do not use a single 1-D dist[v] with Dijkstra — it prunes the few-stops path that you actually need (try the k = 1 example).
- INT_MAX + w would overflow, but here cost always comes from a real popped state, so it is safe.

Code:

cheapest_flights_within_k_stops.cpp

```cpp
class Solution {
public:
    int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k) {
        vector<vector<pair<int,int>>> adj(n);
        for(auto& f : flights) adj[f[0]].push_back({f[1], f[2]});

        // Pair: {cost, node, stops}
        priority_queue<tuple<int,int,int>, vector<tuple<int,int,int>>, greater<>> pq;
        pq.push({0, src, 0});

        vector<vector<int>> dist(n, vector<int>(k + 2, INT_MAX));
        dist[src][0] = 0;

        while(!pq.empty()) {
            auto [cost, u, stops] = pq.top(); pq.pop();
            if(u == dst) return cost;
            if(stops > k) continue;

            for(auto& [v, w] : adj[u]) {
                if(cost + w < dist[v][stops + 1]) {
                    dist[v][stops + 1] = cost + w;
                    pq.push({cost + w, v, stops + 1});
                }
            }
        }
        return -1;
    }
};
```

DRY-RUN:

Input: `n = 4, flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src = 0, dst = 3, k = 1`

Dijkstra on (node, stops) states

Step	What happens
1	Push {0, 0, 0}, dist[0][0] = 0. Here stops counts edges taken, so at most k + 1 = 2 flights are allowed.
2	Pop {0, 0, 0} (cheapest in the heap). Try every flight out of 0.
3	Flight 0→1 ($100): 0 + 100 = 100 < dist[1][1] → store it and push {100, 1, 1}.
4	Pop {100, 1, 1} (cheapest in the heap). Try every flight out of 1.
5	Flight 1→2 ($100): 100 + 100 = 200 < dist[2][2] → store it and push {200, 2, 2}.
6	Flight 1→3 ($600): 100 + 600 = 700 < dist[3][2] → store it and push {700, 3, 2}.
7	Pop {200, 2, 2}: stops = 2 > k = 1 → already used all allowed flights, continue.
8	Pop {700, 3, 2}: u == dst → the heap is ordered by cost, so this is the cheapest valid route. Answer = 700.

Analysis:

- **Time:** O(K * E)
- **Space:** O(V * K)

Contest tips

- The most common accepted solution is Bellman–Ford limited to k+1 rounds (copy the dist array each round!) — O(k·E), very short to code.
- BFS by levels (queue of {node, cost}, at most k+1 levels) with a 1-D dist also works since we only care about level count.
- Same “expand the state” trick: 1293 Shortest Path with Obstacle Elimination, 864 Shortest Path to Get All Keys, 2093 Min Cost to Reach City With Discounts.

WORD LADDER

Problem Statement:

127. Word Ladder

Hard | LeetCode | Open the full problem

A **transformation sequence** from `beginWord` to `endWord` using a dictionary `wordList` is a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Every two adjacent words in the sequence differ by exactly one letter.
- Every s_i for `1 <= i <= k` is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- s_k `== endWord`.

Given `beginWord`, `endWord` and `wordList`, return the **number of words** in the **shortest** transformation sequence from `beginWord` to `endWord`, or `0` if no such sequence exists.

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList =
["hot","dot","dog","lot","log","cog"]
Output: 5
Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" ->
"cog", which contains 5 words.
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
Output: 0
Explanation: endWord "cog" is not in wordList, so no valid transformation sequence
exists.
```

Constraints:

- `1 <= beginWord.length <= 10`
- `endWord.length == beginWord.length`
- `1 <= wordList.length <= 5000`
- `wordList[i].length == beginWord.length`
- `beginWord, endWord, and wordList[i] consist of lowercase English letters.`
- `beginWord != endWord`
- `All the words in wordList are unique.`

Approach:

Pattern: BFS on an implicit graph

- **BFS Selection:** Since we need the shortest path in an unweighted scenario (each word change costs 1), BFS is optimal.
- **Dictionary Check:** Use an unordered_set for O(1) lookups to see if a generated word exists in the list.
- **Transformation:** For the current word, try changing each character from 'a' to 'z'. If the new word is in the set, add it to the queue and remove it from the set (to mark as visited).

Key idea

- Words are nodes, “differ by one letter” is an edge. Every edge costs 1 → BFS gives the shortest sequence.

- Never build the graph explicitly. From a word, generate all L × 26 one-letter variants and look them up in a hash set.
- Erasing a word from st the moment it is pushed doubles as the visited array — each word enters the queue at most once.

Common mistakes

- The answer counts words, not moves — that is why the start is pushed with steps = 1.
- If endWord is not in wordList the answer is 0 (the BFS handles it, but you can return early).
- Restore word[i] = original after the inner loop, otherwise later positions mutate the wrong word.
- Marking visited when popping instead of pushing lets the same word be queued many times → TLE.

Code:

word_ladder.cpp

```cpp
class Solution {
public:
    int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
        // Queue stores {currentWord, steps}
        queue<pair<string, int>> q;
        q.push({beginWord, 1});

        // Hash set for O(1) word discovery
        unordered_set<string> st(wordList.begin(), wordList.end());
        st.erase(beginWord);

        while(!q.empty()) {
            string word = q.front().first;
            int steps = q.front().second;
            q.pop();

            if(word == endWord) return steps;

            for(int i = 0; i < word.size(); i++) {
                char original = word[i];
                for(char ch = 'a'; ch <= 'z'; ch++) {
                    word[i] = ch;
                    // If transformed word exists in dictionary
                    if(st.find(word) != st.end()) {
                        st.erase(word); // Mark as visited
                        q.push({word, steps + 1});
                    }
                }
                word[i] = original; // Backtrack for next character position
            }
        }
        return 0;
    }
};
```

DRY-RUN:

Input: `beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]`

BFS over words, erasing from the set when discovered

Step	What happens
1	Push {"hit", 1}; put the dictionary into st and erase beginWord from it.
2	Pop {"hit", 1}. Try replacing each position i with 'a'..'z' and look the result up in st.
3	i = 1, ch = 'o' → "hot" is in st: erase it (visited) and push {"hot", 2}.

4	Pop {"hot", 2}. Try replacing each position i with 'a'..'z' and look the result up in st.
5	i = 0, ch = 'd' → "dot" is in st: erase it (visited) and push {"dot", 3}.
6	i = 0, ch = 'l' → "lot" is in st: erase it (visited) and push {"lot", 3}.
7	Pop {"dot", 3}. Try replacing each position i with 'a'..'z' and look the result up in st.
8	i = 2, ch = 'g' → "dog" is in st: erase it (visited) and push {"dog", 4}.
9	Pop {"lot", 3}. Try replacing each position i with 'a'..'z' and look the result up in st.
10	i = 2, ch = 'g' → "log" is in st: erase it (visited) and push {"log", 4}.
11	Pop {"dog", 4}. Try replacing each position i with 'a'..'z' and look the result up in st.
12	i = 0, ch = 'c' → "cog" is in st: erase it (visited) and push {"cog", 5}.
13	Pop {"log", 4}. Try replacing each position i with 'a'..'z' and look the result up in st.
14	No variant of "log" is still in st (its neighbours were already discovered) → nothing pushed.
15	Pop "cog": word == endWord → Answer = 5 words (hit → hot → dot → dog → cog).

Analysis:

- **Time:** O(N * M * 26)
- **Space:** O(N * M)

Contest tips

- Complexity O(N · L · 26) lookups (each costs O(L) for hashing).
- Bidirectional BFS (expand the smaller frontier) is a big speed-up on large dictionaries.
- Word Ladder II (126) needs all shortest paths: BFS to record parents/levels, then DFS back from endWord.
- Same pattern: 433 Minimum Genetic Mutation, 752 Open the Lock.

NUMBER OF WAYS TO ARRIVE AT DESTINATION

Problem Statement:

1976. Number of Ways to Arrive at Destination

Medium | LeetCode | Open the full problem

A city has `n` intersections numbered `0` to `n - 1`, joined by **bidirectional** roads. Every intersection can be reached from every other intersection, and there is at most one road between any two intersections.

You are given an integer `n` and a 2D integer array `roads`, where `roads[i] = [u_i, v_i, time_i]` means there is a road between intersections `u_i` and `v_i` that takes `time_i` minutes to travel.

You want to travel from intersection `0` to intersection `n - 1` in the **shortest amount of time**. Return the **number of different ways** you can do this. Since the answer may be large, return it **modulo** `10^9 + 7`.

Example 1:

```
Input: n = 7, roads =
[[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]
Output: 4
Explanation: The shortest time from intersection 0 to intersection 6 is 7 minutes. The
four ways to achieve it are: 0 -> 6; 0 -> 4 -> 6; 0 -> 1 -> 2 -> 5 -> 6; 0 -> 1 -> 3 -> 5
-> 6.
```

Example 2:

```
Input: n = 2, roads = [[1,0,10]]
Output: 1
Explanation: There is only one way to go from intersection 0 to intersection 1, and it
takes 10 minutes.
```

Constraints:

- `1 <= n <= 200`
- `n - 1 <= roads.length <= n * (n - 1) / 2`
- `roads[i].length == 3`
- `0 <= ui, vi <= n - 1`
- `1 <= timei <= 10^9`
- `ui != vi`
- `There is at most one road connecting any two intersections.`
- `You can reach any intersection from any other intersection.`

Approach:

Pattern: Dijkstra + DP on the shortest-path DAG

- **Dijkstra Foundation:** Standard Dijkstra to find the minimum distance from source to all nodes.
- **Path Counting:** If we find a strictly shorter path to a node, we reset its path count to that of its parent. If we find a path of equal distance, we add the parent's path count to it.

Key idea

- Count shortest paths while running Dijkstra: ways[v] = number of shortest paths from 0 to v.
- Strictly shorter path found → old paths are useless: ways[v] = ways[u]. Equal length → another set of shortest paths: ways[v] += ways[u].

- Correctness relies on popping u only when dist[u] is final (so ways[u] is complete) — hence the stale-entry continue is required here.

Common mistakes

- Without if (dist_ > dist[node]) continue; a stale pop would add ways[node] a second time → over-counting.
- Distances can reach ~2·1011: use long long and an infinity like 1e18 (the "Simpler Version" with int/1e9 overflows on big tests).
- Take % MOD on every addition — the "Simpler Version" never reduces ways, so it overflows on graphs with many equal paths.
- Roads are bidirectional: push both directions.

Code:

robust_version.cpp

```cpp
class Solution {
public:
    int countPaths(int n, vector<vector<int>>& roads) {
        vector<vector<pair<int,int>>> adjList(n);
        const int MOD = 1e9 + 7;
        for(auto it: roads) {
            adjList[it[0]].push_back({it[1],it[2]});
            adjList[it[1]].push_back({it[0],it[2]});
        }

        vector<long long> dist(n,1e18);
        vector<long long> ways(n,0);
        int destination=n-1;
        priority_queue<pair<long long,int>, vector<pair<long long,int>>,
greater<pair<long long,int>>> pq;

        pq.push({0,0});
        dist[0]=0;
        ways[0]=1;
        while(!pq.empty()) {
            auto[dist_,node]=pq.top();
            pq.pop();

            if (dist_ > dist[node]) continue;

            for(auto it: adjList[node]) {
                int adjNode = it.first;
                int edgeWeight = it.second;
                long long newdist=dist_+edgeWeight;
                if(newdist < dist[adjNode]) {
                    dist[adjNode]=newdist;
                    ways[adjNode]=ways[node];
                    pq.push({dist[adjNode],adjNode});
                }
                else if(newdist==dist[adjNode]) {
                    ways[adjNode] = (ways[adjNode]+ways[node])%MOD;
                }
            }
        }
        return ways[destination];
    }
};
```

simpler_version.cpp

```cpp
class Solution {
public:
```

```
    int countPaths(int n, vector<vector<int>>& roads) {
        vector<vector<pair<int,int>>> adjList(n);
        for(auto it: roads) {
            adjList[it[0]].push_back({it[1], it[2]});
            adjList[it[1]].push_back({it[0], it[2]});
        }

        vector<int> dist(n, 1e9);
        vector<int> ways(n, 0);
        int destination = n - 1;

        priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

        dist[0] = 0;
        ways[0] = 1;
        pq.push({0, 0});

        while(!pq.empty()) {
            auto [dist_, node] = pq.top();
            pq.pop();

            if(dist_ > dist[node]) continue;

            for(auto it: adjList[node]) {
                int adjNode = it.first;
                int edgeWeight = it.second;

                if(dist_ + edgeWeight < dist[adjNode]) {
                    dist[adjNode] = dist_ + edgeWeight;
                    ways[adjNode] = ways[node];
                    pq.push({dist[adjNode], adjNode});
                } else if(dist_ + edgeWeight == dist[adjNode]) {
                    ways[adjNode] = ways[adjNode] + ways[node];
                }
            }
        }
        return ways[destination];
    }
};
```

DRY-RUN:

Input: `n = 7, roads = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]`

Dijkstra + ways[] counting (robust long long version)

Step	What happens
1	Push {0, 0}, dist[0] = 0, ways[0] = 1 (the empty path). All other dist = 1e18, ways = 0.
2	Pop {0, 0}: dist[0] = 0 is final and ways[0] = 1 is complete. Relax its neighbours.
3	0→6: newdist = 7 < ∞ → shorter: dist[6] = 7, reset ways[6] = ways[0] = 1, push.
4	0→1: newdist = 2 < ∞ → shorter: dist[1] = 2, reset ways[1] = ways[0] = 1, push.
5	0→4: newdist = 5 < ∞ → shorter: dist[4] = 5, reset ways[4] = ways[0] = 1, push.
6	Pop {2, 1}: dist[1] = 2 is final and ways[1] = 1 is complete. Relax its neighbours.
7	1→2: newdist = 5 < ∞ → shorter: dist[2] = 5, reset ways[2] = ways[1] = 1, push.
8	1→3: newdist = 5 < ∞ → shorter: dist[3] = 5, reset ways[3] = ways[1] = 1, push.
9	Pop {5, 2}: dist[2] = 5 is final and ways[2] = 1 is complete. Relax its neighbours.

10	2→5: newdist = 6 < ∞ → shorter: dist[5] = 6, reset ways[5] = ways[2] = 1, push.
11	Pop {5, 3}: dist[3] = 5 is final and ways[3] = 1 is complete. Relax its neighbours.
12	3→6: newdist = 8 > dist[6] = 7 → longer, ignore.
13	3→5: newdist = 6 == dist[5] → another shortest route: ways[5] += ways[3] → 2.
14	Pop {5, 4}: dist[4] = 5 is final and ways[4] = 1 is complete. Relax its neighbours.
15	4→6: newdist = 7 == dist[6] → another shortest route: ways[6] += ways[4] → 2.
16	Pop {6, 5}: dist[5] = 6 is final and ways[5] = 2 is complete. Relax its neighbours.
17	5→6: newdist = 7 == dist[6] → another shortest route: ways[6] += ways[5] → 4.
18	Pop {7, 6}: dist[6] = 7 is final and ways[6] = 4 is complete. Relax its neighbours.
19	Heap empty. Answer = ways[6] = 4 shortest routes of length 7 (green edges form the shortest-path DAG).

Analysis:

- **Time Complexity:** O(E log V)
- **Space Complexity:** O(V + E)

Contest tips

- Same template counts anything over shortest paths: 1786 Restricted Paths, 2192 / “number of shortest paths” in CSES-style tasks (also track min/max edges on shortest paths).
- For unweighted graphs the same idea works with BFS levels.
- O(E log V) time, O(V + E) memory.

SHORTEST PATH IN BINARY MATRIX

Problem Statement:

1091. Shortest Path in Binary Matrix

Medium | LeetCode | Open the full problem

Given an `n x n` binary matrix `grid`, return the length of the shortest **clear path** in the matrix. If there is no clear path, return `-1`.

A **clear path** is a path from the top-left cell `(0, 0)` to the bottom-right cell `(n - 1, n - 1)` such that:

- Every visited cell on the path has value `0`.
- Each pair of consecutive cells on the path is **8-directionally** connected (they are different cells that share an edge or a corner).

The **length** of a clear path is the number of cells visited along it (including the start and end cells).

Example 1:

```
Input: grid = [[0,1],[1,0]]
Output: 2
```

Example 2:

```
Input: grid = [[0,0,0],[1,1,0],[1,1,0]]
Output: 4
```

Example 3:

```
Input: grid = [[1,0,0],[1,1,0],[1,1,0]]
Output: -1
```

Constraints:

- `n == grid.length`
- `n == grid[i].length`
- `1 <= n <= 100`
- `grid[i][j] is 0 or 1`

Approach:

Pattern: Grid BFS (unit weights, 8 neighbours)

- **8-Way Traversal:** Unlike standard grid problems, we use 8 directions (including diagonals) using the direction arrays.
- **BFS for Shortest Path:** Since all edges have weight 1, BFS naturally finds the shortest distance first.
- **Edge Cases:** Check if the source or destination is blocked (grid value is 1) at the start.

Key idea

- Each move costs 1 → BFS. The first time a cell gets a distance it is already the shortest one.
- Path length counts cells, so the start has length 1 (that is why {1,{0,0}} is pushed).
- Check grid[0][0] and grid[n-1][n-1] up front — a blocked endpoint means -1.

Common mistakes

- Look closely at the direction arrays: (dir_r[5], dir_c[5]) = (0,-1) repeats “left” and the diagonal (1,-1) (down-left) is missing. The 8 offsets should be every (dr,dc) ≠ (0,0) with dr,dc ∈ {-1,0,1}. In this example cell (3,0) is only reachable down-left and is never reached.
- dist[0][0] = 0 but its queue entry carries 1. For n == 1 (grid = [[0]]) this returns 0 while the expected answer is 1. Setting dist[0][0] = 1 fixes both.
- Marking visited when popping instead of pushing lets cells be queued many times.

Code:

shortest_path_in_binary_matrix.cpp

```cpp
class Solution {
public:
    int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
        int n=grid.size();
        vector<vector<int>> dist(n,vector<int> (n,1e9));

        if(grid[0][0]==1 || grid[n-1][n-1]==1)
        return -1;

        queue<pair<int,pair<int,int>>> q;
        // {dist,{row,col}}
        q.push({1,{0,0}});
        dist[0][0]=0;

        int dir_r[] = {-1,0,1,0,-1,0,-1,1};
        int dir_c[] = {0,1,0,-1,-1,-1,1,1};

        while(!q.empty())
        {
            auto it=q.front();
            int dist_=it.first;
            int row=it.second.first;
            int col=it.second.second;
            q.pop();

            // traverse the neighbors in all 8 directions
            for(int i=0;i<8;i++)
            {
                int newr = row+dir_r[i];
                int newc = col+dir_c[i];
                if(newr>=0 && newr<n && newc>=0 && newc<n
                && grid[newr][newc]==0 && dist_+1<dist[newr][newc])
                {
                    dist[newr][newc]=dist_+1;
                    q.push({dist_+1,{newr,newc}});
                }
            }
        }
        return dist[n-1][n-1]==1e9? -1: dist[n-1][n-1];
    }
};
```

DRY-RUN:

Input: `grid = [[0,0,1,0],[1,0,1,0],[1,0,0,1],[0,1,0,0]]`

8-direction BFS with a dist matrix

Step	What happens
1	Both corners are 0, so start. Push {1,{0,0}} and set dist[0][0] = 0 (note: the queue carries 1, the matrix stores 0).

2	Pop {1,{0,0}} and scan the 8 offsets in dir_r/dir_c order. Open cells with dist_ + 1 = 2 < dist: (0,1), (1,1) → set dist and push.
3	Pop {2,{0,1}} and scan the 8 offsets in dir_r/dir_c order. No neighbour improves → nothing pushed.
4	Pop {2,{1,1}} and scan the 8 offsets in dir_r/dir_c order. Open cells with dist_ + 1 = 3 < dist: (2,1), (2,2) → set dist and push.
5	Pop {3,{2,1}} and scan the 8 offsets in dir_r/dir_c order. Open cells with dist_ + 1 = 4 < dist: (3,2) → set dist and push. Down-left (3,0) is open but not tried — (1,-1) is missing from the direction arrays.
6	Pop {3,{2,2}} and scan the 8 offsets in dir_r/dir_c order. Open cells with dist_ + 1 = 4 < dist: (1,3), (3,3) → set dist and push.
7	Pop {4,{3,2}} and scan the 8 offsets in dir_r/dir_c order. No neighbour improves → nothing pushed.
8	Pop {4,{1,3}} and scan the 8 offsets in dir_r/dir_c order. Open cells with dist_ + 1 = 5 < dist: (0,3) → set dist and push.
9	Pop {4,{3,3}} and scan the 8 offsets in dir_r/dir_c order. No neighbour improves → nothing pushed.
10	Pop {5,{0,3}} and scan the 8 offsets in dir_r/dir_c order. No neighbour improves → nothing pushed.
11	Queue empty. Answer = dist[3][3] = 4. (Cell (3,0) stays ∞ only because of the missing down-left direction — it does not affect this answer.)

Analysis:

- **Time Complexity:** $O(N^2)$
- **Space Complexity:** $O(N^2)$

Contest tips

- Generate directions with two loops for dr in -1..1, for dc in -1..1, skip (0,0) — impossible to typo.
- Since dist only grows by 1 per level, you can return as soon as you pop (n-1, n-1).
- A* with Chebyshev distance or bidirectional BFS are optional speed-ups. Related: 1293 (obstacle elimination), 542 01 Matrix (multi-source BFS).

FIND THE CITY WITH THE SMALLEST NUMBER OF NEIGHBORS

Problem Statement:

1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

Medium | LeetCode | Open the full problem

There are `n` cities numbered `0` to `n - 1`. You are given an array `edges`, where `edges[i] = [from_i, to_i, weight_i]` is a **bidirectional** weighted edge between cities `from_i` and `to_i`, and an integer `distanceThreshold`.

Return the city that can reach the **smallest** number of other cities through some path whose total distance is **at most** `distanceThreshold`. If several cities tie, return the one with the **greatest** number.

The distance of a path between cities `i` and `j` is the sum of the weights of the edges along that path.

Example 1:

```
Input: n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4
Output: 3
Explanation: Neighboring cities within distance 4 of each city: City 0 -> [City 1, City 2]; City 1 -> [City 0, City 2, City 3]; City 2 -> [City 0, City 1, City 3]; City 3 -> [City 1, City 2]. Cities 0 and 3 each have 2 neighbors within the threshold, so return city 3, the one with the greater number.
```

Example 2:

```
Input: n = 5, edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]], distanceThreshold = 2
Output: 0
Explanation: Neighboring cities within distance 2 of each city: City 0 -> [City 1]; City 1 -> [City 0, City 4]; City 2 -> [City 3, City 4]; City 3 -> [City 2, City 4]; City 4 -> [City 1, City 2, City 3]. City 0 has only 1 neighbor within the threshold.
```

Constraints:

- `2 <= n <= 100`
- `1 <= edges.length <= n * (n - 1) / 2`
- `edges[i].length == 3`
- `0 <= fromi < toi < n`
- `1 <= weighti, distanceThreshold <= 10^4`
- `All pairs (fromi, toi) are distinct.`

Approach:

Pattern: All-pairs shortest paths + filtering

- **Approach A (Floyd-Warshall):** Maintain a 2D matrix. Iterate through all triplets
  (
  k
  ,
  i
  ,
  j

)
(k,i,j) to relax distances. Best for dense graphs.

- **Approach B (Dijkstra):** Convert the edge list to an adjacency list. For every city (source), run a Priority Queue-based Dijkstra to find distances to all other nodes.
- **Filtering:** In both cases, once the shortest path matrix is populated, count neighbors for each city where distance <= threshold.
- **Result:** Return the largest city index among those with the minimum reachable neighbor count.

Key idea

- We need distances from every city → all-pairs shortest paths. n ≤ 100, so $O(n^3)$ Floyd–Warshall is ~10^6 operations: trivial.
- Floyd–Warshall DP: after processing k, dist[i][j] is the shortest path using only intermediate nodes 0..k. That is why k must be the outermost loop.
- Tie-break "greatest index" for free: scan cities in increasing order and update on cnt <= cntCity.

Common mistakes

- Putting k as the inner loop is a classic silent bug — results look plausible but are wrong.
- INT_MAX + w overflows to negative; the == INT_MAX → continue guard (or using 1e9 as ∞) prevents it.
- Edges are undirected: set both dist[u][v] and dist[v][u]. Do not count the city itself (city != adjCity).

Code:

1_floyd_warshall_implementation.cpp

```cpp
class Solution {
public:
    int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {
        vector<vector<int>> dist(n, vector<int> (n, INT_MAX));

        for(int i = 0; i < n; i++) dist[i][i] = 0;

        for(auto it : edges) {
            dist[it[0]][it[1]] = it[2];
            dist[it[1]][it[0]] = it[2];
        }

        for(int k = 0; k < n; k++) {
            for(int i = 0; i < n; i++) {
                for(int j = 0; j < n; j++) {
                    if(dist[i][k] == INT_MAX || dist[k][j] == INT_MAX)
                        continue;

                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }

        int cntCity = INT_MAX;
        int cityNum = 0;

        for(int city = 0; city < n; city++) {
            int cnt = 0;
            for(int adjCity = 0; adjCity < n; adjCity++) {
                if(city != adjCity && dist[city][adjCity] <= distanceThreshold) {
                    cnt++;
                }
            }
            if(cnt <= cntCity) {
                cntCity = cnt;
                cityNum = city;
```

```
            }
        }
        return cityNum;
    }
};
```

find_the_city_with_the_smallest_.cpp

```cpp
class Solution {
public:
    vector<int> dijkstra(int src,int n, vector<vector<pair<int,int>>> adjList)
    {
        vector<int> dist(n,INT_MAX);
        priority_queue<pair<int,int>, vector<pair<int,int>>,
        greater<pair<int,int>>> pq;

        // {dist,node}
        pq.push({0,src});
        dist[src]=0;
        while(!pq.empty())
        {
            auto it=pq.top();
            int node=it.second;
            int dist_=it.first;
            pq.pop();

            for(auto it: adjList[node])
            {
                int adjNode=it.first;
                int edgeWeight=it.second;

                if(dist_+edgeWeight < dist[adjNode])
                {
                    dist[adjNode]= dist_+edgeWeight;
                    pq.push({dist[adjNode],adjNode});
                }
            }
        }
        return dist;
    }

    int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {
        vector<vector<int>> dist(n, vector<int> (n,INT_MAX));

        for(int i=0;i<n;i++)
        dist[i][i]=0;

        vector<vector<pair<int,int>>> adjList(n);

        for(auto it: edges)
        {
            adjList[it[0]].push_back({it[1],it[2]});
            adjList[it[1]].push_back({it[0],it[2]});
        }

        vector<int> dist_;
        for(int src=0;src<n;src++)
        {
            dist_ = dijkstra(src,n,adjList);
            dist[src]=dist_;
        }
        int cntCity=INT_MAX;
        int cityNum=0;

        for(int city=0;city<n;city++)
        {
            int cnt=0;
```

```
60              for(int adjCity=0;adjCity<n;adjCity++)
61              {
62                  if(city!= adjCity &&
63                  dist[city][adjCity]<=distanceThreshold)
64                  {
65                      cnt++;
66                  }
67              }
68              if(cnt<=cntCity)
69                  {
70                      cntCity=cnt;
71                      cityNum= city;
72                  }
73          }
74          return cityNum;
75      }
76  };
```

DRY-RUN:

Input: `n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]], distanceThreshold = 4`

Approach A — Floyd–Warshall, then count reachable cities

Step	What happens
1	Initialise dist: 0 on the diagonal, the edge weight for direct roads (both directions), INT_MAX (∞) elsewhere.
2	k = 0: now allow city 0 as an intermediate stop. Try every pair (i, j) via dist[i][0] + dist[0][j].
3	k = 0 improves nothing (every path through 0 is no shorter).
4	k = 1: now allow city 1 as an intermediate stop. Try every pair (i, j) via dist[i][1] + dist[1][j].
5	k=1, i=0, j=2: dist[0][1] + dist[1][2] = 3 + 1 = 4 < ∞ → update.
6	k=1, i=0, j=3: dist[0][1] + dist[1][3] = 3 + 4 = 7 < ∞ → update.
7	k=1, i=2, j=0: dist[2][1] + dist[1][0] = 1 + 3 = 4 < ∞ → update.
8	k=1, i=3, j=0: dist[3][1] + dist[1][0] = 4 + 3 = 7 < ∞ → update.
9	k = 2: now allow city 2 as an intermediate stop. Try every pair (i, j) via dist[i][2] + dist[2][j].
10	k=2, i=0, j=3: dist[0][2] + dist[2][3] = 4 + 1 = 5 < 7 → update.
11	k=2, i=1, j=3: dist[1][2] + dist[2][3] = 1 + 1 = 2 < 4 → update.
12	k=2, i=3, j=0: dist[3][2] + dist[2][0] = 1 + 4 = 5 < 7 → update.
13	k=2, i=3, j=1: dist[3][2] + dist[2][1] = 1 + 1 = 2 < 4 → update.
14	k = 3: now allow city 3 as an intermediate stop. Try every pair (i, j) via dist[i][3] + dist[3][j].
15	k = 3 improves nothing (every path through 3 is no shorter).
16	City 0: cities with dist ≤ 4 → {1, 2}, cnt = 2. cnt <= cntCity → cityNum = 0 (<= makes a later, larger index win ties).
17	City 1: cities with dist ≤ 4 → {0, 2, 3}, cnt = 3. 3 > cntCity = 2 → keep city 0.
18	City 2: cities with dist ≤ 4 → {0, 1, 3}, cnt = 3. 3 > cntCity = 2 → keep city 0.
19	City 3: cities with dist ≤ 4 → {1, 2}, cnt = 2. cnt <= cntCity → cityNum = 3 (<= makes a later, larger index win

	ties).
20	Answer = 3: fewest reachable neighbours, ties broken by the larger index.

Analysis:

- **Floyd-Warshall Time:** $O(N^3)$
- **Dijkstra Time:** O(N * E log V)
- **Space Complexity:** $O(N^2)$
- **Dijkstra Time:** O(V * (E log V))
- **Dijkstra Space:** O(V + E)

Contest tips

- Use Floyd–Warshall when n ≤ ~400 and you need all pairs; use Dijkstra from each node (O(n·E log n)) for large sparse graphs.
- Floyd–Warshall also handles negative edges (no negative cycles); dist[i][i] < 0 afterwards ⇔ negative cycle.
- Related: 1462 Course Schedule IV (transitive closure with booleans), 399 Evaluate Division, 2976 Min Cost to Convert String.

MIN COST TO CONNECT POINTS

Problem Statement:

1584. Min Cost to Connect All Points

Medium | LeetCode | Open the full problem

You are given an array `points` describing integer coordinates on a 2D plane, where `points[i] = [xᵢ, yᵢ]`.

Connecting two points `[xᵢ, yᵢ]` and `[xⱼ, yⱼ]` costs their **Manhattan distance**: `|xᵢ - xⱼ| + |yᵢ - yⱼ|`.

Return the minimum total cost needed so that all points are connected. All points count as connected when there is **exactly one** simple path between every pair of points.

Example 1:

```
Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]
Output: 20
Explanation: One optimal way to connect the points costs 20 in total. With that set of connections, there is a unique path between every pair of points.
```

Example 2:

```
Input: points = [[3,12],[-2,5],[-4,1]]
Output: 18
```

Constraints:

- `1 <= points.length <= 1000`
- `-10^6 <= xi, yi <= 10^6`
- `All pairs (xi, yi) are distinct.`

Approach:

Pattern: Minimum spanning tree on a complete graph

- Every pair of points can be wired → complete graph with n(n-1)/2 Manhattan-distance edges; we want the cheapest set of n-1 edges connecting everything = MST.
- **Cut property:** the cheapest edge leaving the current tree is always safe to take. Prim just repeats that with a min-heap.
- A popped node that is already inMST is a stale/heavier bridge — skip it (that is what prevents cycles).

Common mistakes

- Mark inMST when popping, not when pushing — marking on push can lock in a non-minimal bridge.
- Pushing every edge costs O(n² log n) time and O(n²) heap memory; for n = 1000 that is ~10⁶ entries — OK, but tight in memory-limited judges.
- Do not stop at the first pop; keep going until the heap is empty (or count n added nodes and break early).

Code:

c_implementation.cpp

```cpp
class Solution {
public:
    int minCostConnectPoints(vector<vector<int>>& points) {
```

```
        int n = points.size();
        // adjList[i] stores {neighbor_node, weight}
        vector<vector<pair<int,int>>> adjList(n);

        // 1. PRE-PROCESS: Calculate distance between every pair of points
        // In this problem, every point can connect to every other point (Dense Graph).
        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
                int cost = abs(points[i][0] - points[j][0]) +
                           abs(points[i][1] - points[j][1]);
                adjList[i].push_back({j, cost});
                adjList[j].push_back({i, cost});
            }
        }

        // 2. SETUP PRIM'S:
        // Priority Queue (PQ) acts as our 'Shortest Bridge Finder'.
        // It stores {weight, node} and keeps the smallest weight at the top.
        priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

        vector<bool> inMST(n, false); // Tracks nodes already in our 'Ink Blot'
        int totalCost = 0;

        // Start with node 0 (weight 0 because we are already there)
        pq.push({0, 0});

        while(!pq.empty()) {
            auto it = pq.top();
            int cost = it.first;
            int node = it.second;
            pq.pop();

            // If node is already in MST, skip to avoid cycles
            if(inMST[node]) continue;

            // ADD TO MST: This node is now part of our minimum cost network
            inMST[node] = true;
            totalCost += cost;

            // Look at all potential bridges from this new node to its neighbors
            for(auto it : adjList[node]) {
                int adjNode = it.first;
                int edgeWeight = it.second;

                // If neighbor isn't in MST, add bridge to the Priority Queue
                if(!inMST[adjNode]) {
                    pq.push({edgeWeight, adjNode});
                }
            }
        }
        return totalCost;
    }
};
```

DRY-RUN:

Input: `points = [[0,0],[2,2],[3,10],[5,2],[7,0]]`

Prim's: grow the tree from point 0 with a {weight, node} min-heap

Step	What happens
1	Pre-compute all Manhattan costs into adjList. Push {0, 0}: start the “ink blot” at point 0 for free.
2	Pop {0, 0}: cheapest bridge to a new point → add 0 to the tree, totalCost += 0 = 0. Push bridges to points not yet in MST: {4,1} {13,2} {7,3} {7,4}.

3	Pop {4, 1}: cheapest bridge to a new point → add 1 to the tree, totalCost += 4 = 4. Push bridges to points not yet in MST: {9,2} {3,3} {7,4}.
4	Pop {3, 3}: cheapest bridge to a new point → add 3 to the tree, totalCost += 3 = 7. Push bridges to points not yet in MST: {10,2} {4,4}.
5	Pop {4, 4}: cheapest bridge to a new point → add 4 to the tree, totalCost += 4 = 11. Push bridges to points not yet in MST: {14,2}.
6	Pop {7, 3}: point 3 is already inMST → continue (taking it would form a cycle).
7	Pop {7, 4}: point 4 is already inMST → continue (taking it would form a cycle).
8	Pop {7, 4}: point 4 is already inMST → continue (taking it would form a cycle).
9	Pop {9, 2}: cheapest bridge to a new point → add 2 to the tree, totalCost += 9 = 20. Every other point is already in the tree, nothing to push.
10	Pop {10, 2}: point 2 is already inMST → continue (taking it would form a cycle).
11	Pop {13, 2}: point 2 is already inMST → continue (taking it would form a cycle).
12	Pop {14, 2}: point 2 is already inMST → continue (taking it would form a cycle).
13	Heap empty, all 5 points are connected by 4 edges. Answer = 20.

Analysis:

- **What Is A Spanning Tree?:** Connects all nodes with NO cycles
- **What Is An Mst?:** The Spanning Tree with the LOWEST total weight
- **The "Greedy" Choice:** Always pick the shortest available edge next
- **Time Complexity:** $O(N^2 \log N)$
- **Space Complexity:** $O(N^2)$

Contest tips

- For dense graphs, array Prim (keep minDist[v], pick the min by linear scan) is $O(n^2)$ with O(n) memory — the optimal choice for this problem.
- You may break as soon as n nodes are in the tree.
- Related: 1135 Connecting Cities With Minimum Cost, 1168 Optimize Water Distribution (add a virtual node 0).

MIN COST TO CONNECT POINTS

Problem Statement:

1584. Min Cost to Connect All Points

Medium | LeetCode | Open the full problem

You are given an array `points` describing integer coordinates on a 2D plane, where `points[i] = [xᵢ, yᵢ]`.

Connecting two points `[xᵢ, yᵢ]` and `[xⱼ, yⱼ]` costs their **Manhattan distance**: `|xᵢ - xⱼ| + |yᵢ - yⱼ|`.

Return the minimum total cost needed so that all points are connected. All points count as connected when there is **exactly one** simple path between every pair of points.

Example 1:

```
Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]
Output: 20
Explanation: One optimal way to connect the points costs 20 in total. With that set of connections, there is a unique path between every pair of points.
```

Example 2:

```
Input: points = [[3,12],[-2,5],[-4,1]]
Output: 18
```

Constraints:

- `1 <= points.length <= 1000`
- `-10^6 <= xi, yi <= 10^6`
- `All pairs (xi, yi) are distinct.`

Approach:

Pattern: MST by sorting edges + Union-Find

- Sort all edges ascending and take an edge iff its endpoints are in different components (cycle property: the heaviest edge of any cycle is never needed).
- DSU answers “same component?” in ~O(1) amortised with path compression + union by rank.
- **Stop after n - 1 edges:** a spanning tree on n nodes has exactly n-1 edges.

Common mistakes

- edges must be stored as {cost, u, v} (cost first) so the default sort orders by cost.
- Always compare roots (findUltimateParent), never parent[u] == parent[v].
- Here findUltimateParent is called in the check and again inside unionSet — harmless thanks to path compression.
- O(n²) edges for n = 1000 → 500k vectors of size 3: prefer array<int,3> or a struct to save memory/time.

Code:

c_implementation.cpp

```cpp
// 1. DSU CLASS: The "Group Manager"
class DSU {
public:
    vector<int> parent, rank;
    DSU(int n) {
```

```
        parent.resize(n); rank.resize(n, 0);
        for(int i = 0; i < n; i++) parent[i] = i;
    }

    // Path Compression: Flatten the tree so every node points to the 'Big Boss'
    int findUltimateParent(int node) {
        if(parent[node] == node) return node;
        return parent[node] = findUltimateParent(parent[node]);
    }

    // Union by Rank: Keep the tree height small for speed
    void unionSet(int u, int v) {
        u = findUltimateParent(u); v = findUltimateParent(v);
        if(u != v) {
            if(rank[u] < rank[v]) swap(u, v);
            parent[v] = u;
            if(rank[u] == rank[v]) rank[u]++;
        }
    }
};

class Solution {
public:
    int minCostConnectPoints(vector<vector<int>>& points) {
        int n = points.size();
        vector<vector<int>> edges;

        // 2. PRE-PROCESS: Calculate distance between every pair of points
        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
                int cost = abs(points[i][0] - points[j][0]) +
                           abs(points[i][1] - points[j][1]);
                edges.push_back({cost, i, j});
            }
        }

        // 3. SORT: Cheapest bridges first (Kruskal's core strategy)
        sort(edges.begin(), edges.end());

        DSU dsu(n);
        int totalCost = 0, edgesUsed = 0;

        // 4. PROCESS: Pick edges until everything is connected
        for(auto &e : edges) {
            int cost = e[0], u = e[1], v = e[2];

            // Connect only if they are not already in the same group
            if(dsu.findUltimateParent(u) != dsu.findUltimateParent(v)) {
                dsu.unionSet(u, v);
                totalCost += cost;
                if(++edgesUsed == n - 1) break;
            }
        }
        return totalCost;
    }
};
```

DRY-RUN:

Input: `points = [[0,0],[2,2],[3,10],[5,2],[7,0]]`

Kruskal's: sorted edges + DSU (union by rank, path compression)

Step	What happens
1	Build all 10 edges {cost, i, j} and sort them — cheapest first. Every point starts as its own group (parent[i] =

	i).
2	Edge 1-3 (cost 3): roots 1 ≠ 3 → different groups. unionSet: parent[3] = 1 (rank[1] = 1). totalCost = 3, edgesUsed = 1.
3	Edge 0-1 (cost 4): roots 0 ≠ 1 → different groups. unionSet: parent[0] = 1 (rank[1] = 1). totalCost = 7, edgesUsed = 2.
4	Edge 3-4 (cost 4): roots 1 ≠ 4 → different groups. unionSet: parent[4] = 1 (rank[1] = 1). totalCost = 11, edgesUsed = 3.
5	Edge 0-3 (cost 7): findUltimateParent(0) = findUltimateParent(3) = 1 → already connected, building it would close a cycle. Skip.
6	Edge 0-4 (cost 7): findUltimateParent(0) = findUltimateParent(4) = 1 → already connected, building it would close a cycle. Skip.
7	Edge 1-4 (cost 7): findUltimateParent(1) = findUltimateParent(4) = 1 → already connected, building it would close a cycle. Skip.
8	Edge 1-2 (cost 9): roots 1 ≠ 2 → different groups. unionSet: parent[2] = 1 (rank[1] = 1). totalCost = 20, edgesUsed = 4. That is n - 1 = 4 edges → break.
9	Done: 4 edges chosen. Answer = 20 (same MST as Prim's).

Analysis:

- **What Is Kruskal'S?:** A Greedy edge-based MST algorithm
- **Edge Strategy:** Sort ALL edges and pick the cheapest first
- **The "Cycle" Guard:** Use DSU to check if nodes are already connected
- **Time Complexity:** O(N^2 log N)
- **Space Complexity:** O(N^2)

Contest tips

- Kruskal shines on sparse edge lists; on complete graphs Prim O(n^2) is asymptotically better (O(n^2 log n) for the sort here).
- Keep a DSU template ready: it also solves 1319, 684 Redundant Connection, 721 Accounts Merge, 1101 Earliest Moment Everyone Becomes Friends.
- Union by rank alone gives O(log n); combined with path compression it is α(n) — effectively constant.

NUMBER OF OPERATIONS TO MAKE NETWORK CONNECTED

Problem Statement:

1319. Number of Operations to Make Network Connected

Medium | LeetCode | Open the full problem

There are `n` computers numbered `0` to `n - 1`, linked by ethernet cables into a network. You are given `connections`, where `connections[i] = [a_i, b_i]` is a cable between computers a_i and b_i. Any computer can reach any other computer directly or indirectly through the network.

In one operation, you may remove a cable between two directly connected computers and plug it in between any pair of computers that are not directly connected.

Return the **minimum** number of operations needed to make all the computers connected. If it is not possible, return `-1`.

Example 1:

```
Input: n = 4, connections = [[0,1],[0,2],[1,2]]
Output: 1
Explanation: Remove the cable between computers 1 and 2 and place it between computers 1 and 3.
```

Example 2:

```
Input: n = 6, connections = [[0,1],[0,2],[0,3],[1,2],[1,3]]
Output: 2
```

Example 3:

```
Input: n = 6, connections = [[0,1],[0,2],[0,3],[1,2]]
Output: -1
Explanation: There are not enough cables.
```

Constraints:

- `1 <= n <= 10^5`
- `1 <= connections.length <= min(n * (n - 1) / 2, 10^5)`
- `connections[i].length == 2`
- `0 <= ai, bi < n`
- `ai != bi`
- `There are no repeated connections.`
- `No two computers are connected by more than one cable.`

Approach:

Pattern: Connected components with Union-Find

- To join C components you need exactly C - 1 cables. Where do they come from? Only from redundant cables — ones whose endpoints were already connected.
- **So:** answer = C - 1 if cntExtras ≥ C - 1, else -1.
- **Shortcut:** a connected graph needs ≥ n-1 edges, so connections.size() < n - 1 ⇒ -1 immediately; otherwise the answer is always C - 1.

Common mistakes

- Count components as nodes with parent[i] == i — after all unions. Isolated computers (no cables) are components too.
- Compare ultimate parents, not parent[u] directly.
- Rank is a height bound, not a size; do not use it to count members.

Code:

c_implementation.cpp

```cpp
class Solution {
public:
class DisjointSet{
    public:
    int n;
    vector<int> parent;
    vector<int> rank;

    DisjointSet(int n_)
    {
        n=n_;
        parent.resize(n);
        rank.resize(n,0);
        for(int i=0;i<n;i++)
            parent[i]=i; // Initially, every node is its own parent (own group)
    }

    // Path Compression: Flattens the tree for near O(1) lookups
    int findUltimateParent(int node)
    {
        if(node==parent[node])
            return node;

        // Recursively find root and attach current node directly to it
        return parent[node]=findUltimateParent(parent[node]);
    }

    // Union by Rank: Attaches the smaller tree to the larger tree
    void unionBysize(int u,int v)
    {
        int pu = findUltimateParent(u);
        int pv = findUltimateParent(v);
        if(pu==pv) return; // Already in the same group

        if(rank[pu]>rank[pv])
        {
            parent[pv]=pu;
        }
        else if(rank[pv]>rank[pu])
        {
            parent[pu]=pv;
        }
        else
        {
            parent[pv]=pu;
            rank[pu]++; // Increase rank if heights are equal
        }
    }
};

int makeConnected(int n, vector<vector<int>>& connections) {
    DisjointSet ds(n);
    int cntExtras=0; // To store number of redundant cables

    // Step 1: Iterate through each connection
    for(auto it: connections)
    {
```

```
            int u = it[0];
            int v = it[1];

            // If they share the same root, this cable is redundant
            if(ds.findUltimateParent(u)==ds.findUltimateParent(v))
            {
                cntExtras++;
            }
            else
            {
                // Otherwise, merge the two groups
                ds.unionBysize(u,v);
            }
        }

        // Step 2: Count the total number of connected components
        int cntC=0;
        for(int i=0;i<n;i++)
        {
            // A node is a root if it points to itself
            if(ds.parent[i]==i)
                cntC++;
        }

        // Step 3: Check if we have enough extras to connect all components
        // Formula: Edges needed = Components - 1
        int ans=cntC-1;

        if(cntExtras>=ans)
            return ans;

        return -1; // Not enough redundant cables
    }
    };
```

DRY-RUN:

Input: `n = 6, connections = [[0,1],[2,3],[1,3],[3,0],[1,2]]`

DSU: count redundant cables and components

Step	What happens
1	DisjointSet ds(6): every computer is its own group (parent[i] = i, rank = 0).
2	Cable 0-1: roots 0 ≠ 1 → unionBysize: equal ranks → parent[1] = 0, rank[0]++.
3	Cable 2-3: roots 2 ≠ 3 → unionBysize: equal ranks → parent[3] = 2, rank[2]++.
4	Cable 1-3: roots 0 ≠ 2 → unionBysize: equal ranks → parent[2] = 0, rank[0]++.
5	Cable 3-0: findUltimateParent(3) = findUltimateParent(0) = 0 → already connected, this cable is spare. cntExtras = 1. Path compression on the way: 3→0.
6	Cable 1-2: findUltimateParent(1) = findUltimateParent(2) = 0 → already connected, this cable is spare. cntExtras = 2.
7	Count roots (parent[i] == i): {0, 4, 5} → cntC = 3 separate networks. We need ans = cntC - 1 = 2 moves.
8	cntExtras = 2 ≥ ans = 2 → enough spare cables. Answer = 2.

Analysis:

- **Core Strategy:** Identify Isolated Groups & Excess Wires

- **The Requirement:** N-1 edges are needed for N nodes
- **The Formula:** Operations = (Total Components - 1)
- **Time Complexity:** O(E + N)
- **Space Complexity:** O(N)

Contest tips

- DFS/BFS component counting works equally well: answer = components - 1 after the n-1 edge check.
- O((n + E) α(n)).
- Related: 547 Number of Provinces, 684 Redundant Connection, 990 Satisfiability of Equality Equations, 1579 Remove Max Number of Edges.

STRONGLY CONNECTED COMPONENTS (KOSARAJU'S ALGORITHM)

Problem Statement:

Strongly Connected Components (Kosaraju's Algorithm)

Medium | GeeksforGeeks | Open the full problem

You are given a **directed** graph with `V` vertices numbered `0` to `V - 1`, as an adjacency list `adj`, where `adj[u]` lists every vertex `v` such that there is a directed edge `u -> v`.

A **strongly connected component** (SCC) is a maximal set of vertices in which every vertex can reach every other vertex of the set by following directed edges. Every vertex belongs to exactly one SCC (a vertex with no cycle through it forms an SCC by itself).

Return the **number** of strongly connected components in the graph.

Example 1:

```
Input: V = 5, adj = [[2,3],[0],[1],[4],[]]
Output: 3
Explanation: Edges: 0 -> 2, 0 -> 3, 1 -> 0, 2 -> 1, 3 -> 4. The SCCs are {0, 1, 2} (0 -> 2 -> 1 -> 0 is a cycle), {3} and {4}, so there are 3.
```

Example 2:

```
Input: V = 3, adj = [[1],[2],[0]]
Output: 1
Explanation: Edges 0 -> 1 -> 2 -> 0 form a single cycle, so all three vertices are in one SCC.
```

Constraints:

- `1 <= V <= 5000`
- `0 <= E <= V * (V - 1)`
- `0 <= u, v <= V - 1`
- `Expected time complexity: O(V + E)`
- `Expected auxiliary space: O(V + E)`

Approach:

Pattern: Strongly connected components (two-pass DFS)

- Pass 1 records finish order. The node that finishes last lies in a “source” SCC of the condensation DAG.
- Transposing reverses every edge but keeps each SCC intact. A source SCC becomes a sink, so DFS from it on the transposed graph cannot leak into other SCCs.
- Popping the stack visits SCCs in topological order of the original condensation; every DFS that starts in pass 2 is exactly one SCC. Anything it could leak into was already visited.

Common mistakes

- Push to the stack after exploring all children (post-order). Pushing on entry breaks the algorithm.
- Pass 2 must use the transposed graph and a fresh visited array.
- Deep recursion on $V = 10^5$ chains can overflow the stack; use an iterative DFS or raise the stack size.

Code:

c_implementation.cpp

```cpp
// Standard DFS to store nodes in stack according to finish time
void dfs(int node, vector<vector<int>> &adj, vector<int> &visited, stack<int> &st) {
    visited[node] = 1;
    for (auto it : adj[node]) {
        if (!visited[it]) {
            dfs(it, adj, visited, st);
        }
    }
    // Push to stack after visiting all children (Post-order)
    st.push(node);
}

// Second DFS to traverse the reversed (transposed) graph
void dfs2(int node, vector<vector<int>> &adj, vector<int> &visited) {
    visited[node] = 1;
    for (auto it : adj[node]) {
        if (!visited[it]) {
            dfs2(it, adj, visited);
        }
    }
}

int kosaraju(int V, vector<vector<int>> &adj) {
    stack<int> st;
    vector<int> visited(V, 0);

    // 1. Fill the stack based on finish times
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(i, adj, visited, st);
        }
    }

    // 2. Transpose the graph (Reverse all edge directions)
    vector<vector<int>> transposedAdj(V);
    for (int i = 0; i < V; i++) {
        for (auto it : adj[i]) {
            transposedAdj[it].push_back(i);
        }
    }

    // 3. Perform DFS on the transposed graph in Stack order
    vector<int> visited2(V, 0);
    int sccCount = 0;

    while (!st.empty()) {
        int node = st.top();
        st.pop();

        if (!visited2[node]) {
            // This marks the discovery of a new SCC
            dfs2(node, transposedAdj, visited2);
            sccCount++;
        }
    }
    return sccCount;
}
```

DRY-RUN:

Input: `V = 8, edges: 0→1, 1→2, 2→0, 1→3, 3→4, 4→5, 5→3, 6→5, 6→7, 7→6`

Kosaraju: finish-order DFS → transpose → DFS in stack order

Step	What happens

1	Step 1: DFS the original graph from every unvisited node; push a node onto st only when all its neighbours are done.
2	dfs(0): mark visited. Neighbours in adj[0]: [1].
3	dfs(1): mark visited. Neighbours in adj[1]: [2, 3].
4	dfs(2): mark visited. Neighbours in adj[2]: [0].
5	All neighbours of 2 finished → st.push(2) (post-order).
6	dfs(3): mark visited. Neighbours in adj[3]: [4].
7	dfs(4): mark visited. Neighbours in adj[4]: [5].
8	dfs(5): mark visited. Neighbours in adj[5]: [3].
9	All neighbours of 5 finished → st.push(5) (post-order).
10	All neighbours of 4 finished → st.push(4) (post-order).
11	All neighbours of 3 finished → st.push(3) (post-order).
12	All neighbours of 1 finished → st.push(1) (post-order).
13	All neighbours of 0 finished → st.push(0) (post-order).
14	dfs(6): mark visited. Neighbours in adj[6]: [5, 7].
15	dfs(7): mark visited. Neighbours in adj[7]: [6].
16	All neighbours of 7 finished → st.push(7) (post-order).
17	All neighbours of 6 finished → st.push(6) (post-order).
18	Step 2: build transposedAdj — every edge u→v becomes v→u. SCCs stay the same, but edges between SCCs now point backwards. Stack top is 6 (finished last).
19	Step 3: pop 6 — not in visited2, so it starts a new SCC.
20	dfs2(6) on the transposed graph: neighbours [7].
21	dfs2(7) on the transposed graph: neighbours [6] — already-visited ones (6) belong to earlier SCCs or this one, so DFS cannot leak.
22	dfs2 returned having visited {6, 7} — that is one SCC. sccCount = 1.
23	Pop 7: already in an SCC → skip.
24	Step 3: pop 0 — not in visited2, so it starts a new SCC.
25	dfs2(0) on the transposed graph: neighbours [2].
26	dfs2(2) on the transposed graph: neighbours [1].
27	dfs2(1) on the transposed graph: neighbours [0] — already-visited ones (0) belong to earlier SCCs or this one, so DFS cannot leak.
28	dfs2 returned having visited {0, 1, 2} — that is one SCC. sccCount = 2.
29	Pop 1: already in an SCC → skip.
30	Step 3: pop 3 — not in visited2, so it starts a new SCC.
31	dfs2(3) on the transposed graph: neighbours [1, 5] — already-visited ones (1) belong to earlier SCCs or this

	one, so DFS cannot leak.
32	dfs2(5) on the transposed graph: neighbours [4, 6] — already-visited ones (6) belong to earlier SCCs or this one, so DFS cannot leak.
33	dfs2(4) on the transposed graph: neighbours [3] — already-visited ones (3) belong to earlier SCCs or this one, so DFS cannot leak.
34	dfs2 returned having visited {3, 4, 5} — that is one SCC. sccCount = 3.
35	Pop 4: already in an SCC → skip.
36	Pop 5: already in an SCC → skip.
37	Pop 2: already in an SCC → skip.
38	Stack empty. Answer = 3 strongly connected components: {6,7}, {0,1,2}, {3,4,5}.

Analysis:

- **Core Logic:** Three-Step Graph Reversal
- **Primary Tool:** DFS & Stack (Finish Time)
- **Key Benefit:** Linear time discovery of clusters
- **Time Complexity:** O(V + E)
- **Space Complexity:** O(V + E)

Contest tips

- O(V + E) time and memory. Tarjan's algorithm does it in one pass with low-link values — worth knowing for bridges/articulation points too.
- Condensing SCCs into a DAG is the standard first step for 2-SAT, "minimum edges to make strongly connected" (max(#sources, #sinks)), and DP on cyclic graphs.
- Related: 1192 Critical Connections (Tarjan), 2360 Longest Cycle, GfG "Strongly Connected Components (Kosaraju)".